Sail Code Book

Understanding the Sail Query Engine
at the Code Level

Alexy Khrabrov
chiefscientist.org

Claude Code

Contents

Preface

What Is Sail?

Sail is a query engine that speaks Spark's language. It implements the Apache Spark Connect gRPC protocol in 100% Rust, meaning that any code written for PySpark — every `DataFrame.groupBy`, every `spark.sql(...)`, every UDF — can run against Sail without modification. There is no JVM. There is no Scala runtime. There is no garbage collector pausing your workload at 200 GB/s to decide what to clean up.

The pitch is credible because Sail is not just a SQL engine bolted behind a Spark-shaped facade. It leans on Apache Arrow as its in-memory data format and Apache DataFusion as its query planning and execution backbone — two of the most production-hardened Rust data-processing libraries available. What Sail adds on top is the thick layer of Spark-specific semantics: the precise type-coercion rules, the nullable-by-default schema, the `coalesce` / `repartition` / `range` operators, the Spark session model, the catalog abstractions that connect to AWS Glue, Hive Metastore, Apache Iceberg, Delta Lake, Databricks Unity Catalog, and Microsoft OneLake.

The result is a binary that starts in milliseconds, fits in a Docker image smaller than the JVM alone, and has been benchmarked at roughly 4× faster than Apache Spark on TPC-H with 94% lower infrastructure cost.

Who This Book Is For

This book is for engineers who want to understand how Sail works at the code level — not just how to use it, but why it is built the way it is, where the interesting decisions live, and how to extend or contribute to it.

The assumed reader is comfortable with Rust: ownership, lifetimes, traits, async/await, and the tokio executor. You do not need to be a Spark veteran; Spark concepts are explained where they differ from what a DataFusion or Arrow user might expect. You do not need prior experience with gRPC, Protobuf, or Arrow Flight; those are introduced in context.

The book is *not* a user guide. For installation, configuration, and PySpark compatibility tables, see the Sail documentation.

How to Read This Book

Each chapter is self-contained but builds on the previous one. The path through a query — from PySpark client to Arrow bytes on the wire — is the organizing spine:

- **Chapter 1** gives the 10,000-foot view: how all the pieces connect.
- **Chapters 2–5** trace a query from the moment it enters the gRPC server through logical planning, optimization, physical execution, and result delivery.
- **Chapters 6–7** go deeper into the execution engine and the catalog layer, which are the parts most likely to need extension.
- **Chapter 8** collects Rust patterns that appear throughout the codebase — the actor model, error propagation, code generation, the PyO3 bridge — explained in one place.
- **Chapter 9** closes with how to navigate the codebase and contribute.

Code is quoted directly from the repository. File paths are given relative to the repository root (`crates/sail-spark-connect/src/server.rs`) so you can follow along. Version at time of writing: **0.6.3**.

If you want to skip to a specific subsystem:

"I want to understand…"	Start at
The Spark-to-Sail entry point	Chapter 2
How Arrow flows through the system	Chapter 3
DataFusion extension points	Chapter 4
How results stream back to Python	Chapter 5
The distributed executor	Chapter 6
Adding a new catalog	Chapter 7
Rust architectural patterns	Chapter 8

Chapter 1: Architecture Overview

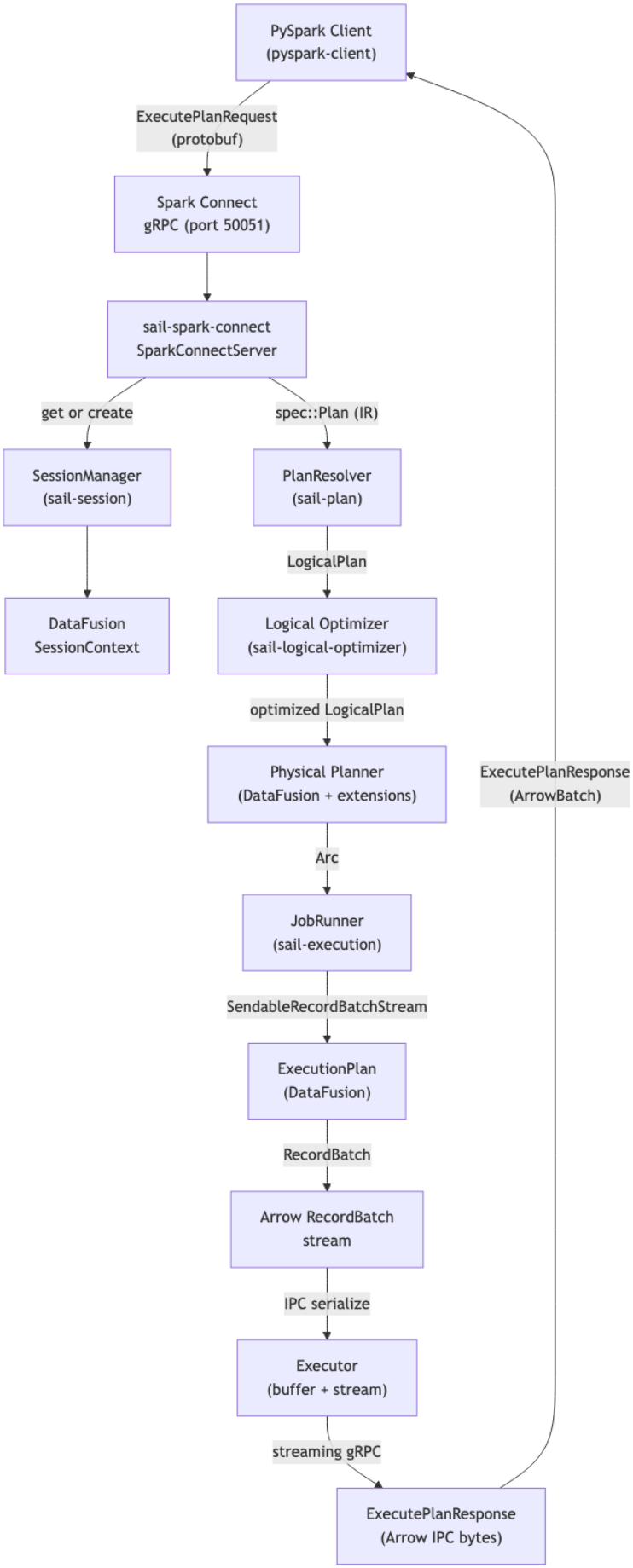
The Problem Space

Apache Spark is the de-facto standard for large-scale data processing. Its DataFrame API and Spark SQL dialect are understood by millions of engineers, embedded in thousands of pipelines, and wrapped by dozens of downstream tools. The problem is not the API — the problem is the runtime. The JVM carries significant overhead: slow startup, non-deterministic GC pauses, memory management via off-heap tricks, and a deployment model that requires spinning up JVM processes on every worker.

Sail's thesis is that the Spark API is worth keeping and the Spark runtime is worth replacing. The Spark Connect protocol, introduced in Spark 3.4, makes that substitution possible: it separates the client (PySpark) from the server with a well-defined gRPC boundary. A client that speaks Spark Connect can be pointed at any conforming server — including one written entirely in Rust.

The Query Path

Here is what happens when a PySpark program executes `df.filter(col("amount") > 100).show()` against a Sail server.


PySpark Client
(pyspark-client)
ExecutePlanRequest
(protobuf)
Spark Connect
gRPC (port 50051)
sail-spark-connect
SparkConnectServer
get or create
spec::Plan (IR)
SessionManager
(sail-session)
PlanResolver
(sail-plan)
LogicalPlan
DataFusion
SessionContext
Logical Optimizer
(sail-logical-optimizer)
optimized LogicalPlan
Physical Planner
(DataFusion + extensions)
ExecutePlanResponse
(ArrowBatch)
Arc
JobRunner
(sail-execution)
SendableRecordBatchStream
ExecutionPlan
(DataFusion)
RecordBatch
Arrow RecordBatch
stream
IPC serialize
Executor
(buffer + stream)
streaming gRPC
ExecutePlanResponse
(Arrow IPC bytes)

Each box in this diagram corresponds to one or more Rust crates. The hand-off points — the types crossing crate boundaries — are the interesting design choices, and most of this book is about them.

The Crate Landscape

Sail's workspace contains ~35 crates under `crates/`. They fall into a few natural groups.

Protocol and Entry Points

Crate	Role
`sail-spark-connect`	Implements the `SparkConnectService` gRPC trait from the Spark Connect protobuf. This is the primary entry point for PySpark clients.
`sail-flight`	Implements Apache Arrow Flight SQL. An alternative entry point for ADBC/JDBC clients that speak Flight SQL rather than Spark Connect.
`sail-server`	Low-level gRPC server builder, actor system, retry logic. Used by both entry points.
`sail-cli`	The `sail` binary — parses command-line arguments and starts either server.
`sail-python`	PyO3 bindings. The `pysail._native` Python module that starts the embedded Rust server from Python.

Planning

Crate	Role
`sail-sql-parser`	Custom SQL parser built with `chumsky` (Pratt combinator library). Contains its own lexer, token types, AST, and keyword codegen — no `sqlparser-rs` dependency.
`sail-sql-analyzer`	Converts the SQL AST into `sail-common`'s internal `spec` IR: statement-level conversion, type inference, interval/date/timestamp parsing.
`sail-plan`	The main planning crate: `PlanResolver` translates `spec::Plan` → DataFusion `LogicalPlan`, then drives optimization and physical planning. Contains `resolve_and_execute_plan`. Also houses the `ScalarFunctionBuilder` DSL and the 400+ function registry.
`sail-plan-lakehouse`	Bridges write operations (Delta/Iceberg) to physical plans. Contains `ExpandRowLevelOp` logical optimizer rule and `DeltaExtensionPlanner`.
`sail-logical-plan`	17 custom `UserDefinedLogicalNodeCore` implementations: `RangeNode`, `ExplicitRepartitionNode`, `ShowStringNode`, `MapPartitionsNode`, `FileWriteNode`, `BarrierNode`, 5 streaming nodes, and more.
`sail-physical-plan`	Corresponding custom `ExecutionPlan` implementations for all 17 logical nodes.
`sail-logical-optimizer`	Custom logical optimizer rules, e.g. `DecorrelateLateralProjection`.
`sail-physical-optimizer`	Rebuilds the entire physical optimizer pipeline from scratch, embedding all DataFusion rules in order plus custom rules: `JoinReorder` (DP algorithm), `RewriteExplicitRepartition`, `RewriteCollectLeftHashJoin`, `EnforceBarrierPartitioning`.
`sail-session`	Session lifecycle and the physical planning bridge: `ExtensionQueryPlanner`, `ExtensionPhysicalPlanner` (dispatches all 17 custom nodes to their physical counterparts), `SessionFactory`, `SessionManager`.

Execution

Crate	Role
`sail-execution`	The distributed execution engine. Implements `JobRunner` with two backends: `LocalJobRunner` (single-process) and `ClusterJobRunner` (driver/worker cluster). Contains the `DriverActor`, `WorkerActor`, task scheduling, and inter-node stream transport.
`sail-common-datafusion`	Shared DataFusion utilities: `SessionExtension`/`SessionExtensionAccessor` traits, `JobRunner`/`JobService` traits, `TableFormat` trait and `TableFormatRegistry`, catalog display helpers.

Catalogs

Crate	Role
`sail-catalog`	The `CatalogProvider` trait and shared catalog utilities.
`sail-catalog-memory`	In-memory catalog (default).
`sail-catalog-glue`	AWS Glue Data Catalog.
`sail-catalog-hms`	Hive Metastore (Thrift).
`sail-catalog-iceberg`	Apache Iceberg REST catalog.
`sail-catalog-unity`	Databricks Unity Catalog.
`sail-catalog-onelake`	Microsoft OneLake / Fabric.
`sail-catalog-system`	Built-in system catalog (`spark_catalog`).

Table Formats and Data Sources

Crate	Role
`sail-delta-lake`	Delta Lake: read, append/overwrite write, MERGE (via row-level write), Variant Shredding, Deletion Vectors. Basic delete/update route through Delta's physical planner; optimize/vacuum/CDC are not yet implemented.
`sail-iceberg`	Iceberg table read via `iceberg-rust`; write not yet implemented.
`sail-data-source`	File-based sources (Parquet, CSV, JSON, ORC, Avro), schema/compression inference, listing table source.
`sail-object-store`	Object store adapters (S3, GCS, Azure Blob) with Spark-compatible URI schemes.

Supporting

Crate	Role
`sail-common`	Cross-cutting primitives: `spec` IR (2,476 lines, 74 named types), config, error types, datetime utilities.
`sail-function`	Spark function implementations (scalar, aggregate, window, table). Used by `sail-plan`'s function registry.
`sail-sql-macro`	Proc-macros for the SQL parser: `#[derive(TreeParser)]`, `#[derive(TreeSyntax)]`, `#[derive(TreeText)]` — derive recursive-descent parsers and unparser from annotated AST structs.
`sail-python-udf`	PyO3-based Python UDF/UDTF execution; supports batch, Arrow-batch, Pandas scalar, grouped map, co-grouped map UDF types.
`sail-telemetry`	OpenTelemetry tracing and metrics.
`sail-gold-test`	Golden-file test infrastructure: generates test suites from Spark's own function documentation, diffs Sail output against Spark output.
`sail-build-scripts`	Build script utilities for codegen.

The Internal IR: spec

One design choice that shapes the whole codebase is the existence of a crate-internal intermediate representation: the `spec` module in `sail-common`. When `sail-spark-connect` receives a protobuf `Plan`, it does not hand the raw protobuf types to the plan resolver. Instead, it converts them into `spec::Plan`, `spec::QueryNode`, `spec::CommandNode`, `spec::Expr`, etc. — Rust types that capture the same semantics but without the protobuf boilerplate.

The spec IR is larger than it might appear: `spec/plan.rs` is 1,356 lines with 74 named types. The `QueryNode` enum has 50+ variants (Filter, Join, Aggregate, Pivot, Unpivot, GroupMap, CoGroupMap, ApplyInPandasWithState, WithWatermark, StatSampleBy, …); the `CommandNode` enum covers everything from `CreateTable` to `MergeInto` to `AlterColumnType`. This is a full Spark relational algebra IR.

`sail-plan`'s resolver has no `prost` dependency; it works with clean Rust enums. The same `spec` types are used by `sail-flight` and `sail-sql-analyzer`, so all three entry points converge to the same planning code.

Dependency Layering

The crate dependency graph is strictly layered to prevent cycles:

```
sail-python  →  sail-spark-connect  →  sail-plan  →  sail-common
                sail-flight         →  sail-sql-analyzer
                                    →  sail-logical-plan
                                    →  sail-physical-plan
```

→ sail-execution
→ sail-catalog-*

`sail-common` has no dependencies on other Sail crates. `sail-plan` does not import from `sail-spark-connect`. The execution engine (`sail-execution`) is likewise isolated from the protocol layer — it receives a `Arc<dyn ExecutionPlan>` and produces a `SendableRecordBatchStream`; it has no knowledge of Spark Connect.

This layering is enforced by the Cargo workspace. Circular dependencies would cause a compile error.

The Two Runtimes

Sail runs two Tokio runtimes: a *primary* runtime for the server (gRPC, session management, query planning) and a *worker* runtime for execution tasks. The `RuntimeHandle` type in `sail-common` wraps both and is threaded through the codebase wherever a handle to the right runtime is needed. This separation allows the execution layer to be offloaded to its own thread pool without interfering with the server's responsiveness.

When running in embedded Python mode (`SparkConnectServer` via PyO3), there is an additional constraint: the Python GIL. Python UDFs must be able to call back into the Python interpreter, which requires releasing the GIL on the Rust side at the right points. `sail-python/src/spark/server.rs` handles this by running the Tokio server on a dedicated OS thread and using `py.detach(...)` to release the GIL when blocking on server shutdown.

Summary

A PySpark query enters Sail as a protobuf message, is decoded into the `spec` IR, resolved and optimized by DataFusion's planner plus Sail's custom nodes and rules, dispatched to a job runner that produces a RecordBatch stream, serialized into Arrow IPC, and streamed back over gRPC. Every stage of this pipeline is a Rust crate with a clean interface. The rest of this book zooms into each stage in turn.

Chapter 2: The Spark Connect Protocol

Why Spark Connect Matters

Before Spark 3.4, the only way to talk to a Spark cluster from Python was through the Py4J bridge: PySpark would serialize Python objects into JVM objects using a local socket, execute on the JVM, and deserialize results back. This architecture tied the client tightly to the server — same version, same JVM, same cluster. There was no stable binary protocol, no way to implement an alternative server.

Spark Connect changed this. It defines a gRPC service with a protobuf schema that describes Spark's entire logical plan space: every relation type (scan, join, aggregate, window, …), every expression type (literal, column reference, function call, …), every command (write, create view, register UDF, …). The Python client constructs a `Plan` protobuf on the local machine and sends it to the server; the server runs it and streams results back as Arrow IPC. Client and server are now separate processes speaking a documented protocol.

For Sail, this is the foundation. Sail is the server. It does not need to ship with Spark, match Spark's JVM version, or run Scala code. It just needs to implement the `SparkConnectService` gRPC interface faithfully.

The Protobuf Schema

The Spark Connect proto files define several services and message types. The key service:

```
service SparkConnectService {
  rpc ExecutePlan(ExecutePlanRequest) returns (stream ExecutePlanResponse);
  rpc AnalyzePlan(AnalyzePlanRequest) returns (AnalyzePlanResponse);
  rpc Config(ConfigRequest) returns (ConfigResponse);
  rpc AddArtifacts(stream AddArtifactsRequest) returns (AddArtifactsResponse);
  rpc ArtifactStatuses(ArtifactStatusesRequest) returns (ArtifactStatusesResponse);
  rpc Interrupt(InterruptRequest) returns (InterruptResponse);
  rpc ReattachExecute(ReattachExecuteRequest) returns (stream ExecutePlanResponse);
  rpc ReleaseExecute(ReleaseExecuteRequest) returns (ReleaseExecuteResponse);
  rpc ReleaseSession(ReleaseSessionRequest) returns (ReleaseSessionResponse);
  rpc FetchErrorDetails(FetchErrorDetailsRequest) returns
(FetchErrorDetailsResponse);
}
```

Notice that `ExecutePlan` returns a *stream* of responses — this is how large result sets are chunked. `ReattachExecute` is a Spark Connect innovation: if the client loses its connection mid-stream, it can reconnect and resume from where it left off using the `response_id` values in each chunk. This requires the server to buffer recent responses.

In Sail, the protos are compiled at build time with `tonic`'s `include_proto!` macro. The generated types live behind the `crate::spark::connect` path:

```
// crates/sail-spark-connect/src/lib.rs
pub mod spark {
    #[expect(clippy::all, clippy::allow_attributes)]
    pub mod connect {
        tonic::include_proto!("spark.connect");
        tonic::include_proto!("spark.connect.serde");
        pub const FILE_DESCRIPTOR_SET: &[u8] =
            tonic::include_file_descriptor_set!("spark_connect_descriptor");
    }
    #[expect(clippy::doc_markdown)]
    pub mod config {
        include!(concat!(env!("OUT_DIR"), "/spark_config.rs"));
    }
}
```

The `#[expect(clippy::all)]` annotation is necessary because generated code does not follow Sail's strict Clippy configuration (which bans `unwrap`, `expect`, and `panic` in non-generated code).

`SparkConnectServer`: The gRPC Handler

The heart of `sail-spark-connect` is `SparkConnectServer` in `crates/sail-spark-connect/src/server.rs`. It holds a single field — a `SessionManager` — and implements the `SparkConnectService` trait generated from the proto:

```
#[derive(Debug)]
pub struct SparkConnectServer {
    session_manager: SessionManager,
}

#[tonic::async_trait]
impl SparkConnectService for SparkConnectServer {
    type ExecutePlanStream = ExecutePlanResponseStream;

    async fn execute_plan(
        &self,
        request: Request<ExecutePlanRequest>,
```

```
    ) -> Result<Response<Self::ExecutePlanStream>, Status> {
        let request = request.into_inner();
        let session_id = request.session_id;
        let user_id = request.user_context.map(|u| u.user_id).unwrap_or_default();
        let metadata = ExecutorMetadata {
            operation_id: request
                .operation_id
                .unwrap_or_else(|| Uuid::new_v4().to_string()),
            tags: request.tags,
            reattachable: is_reattachable(&request.request_options),
        };
        let ctx = self
            .session_manager
            .get_or_create_session_context(session_id, user_id)
            .await
            .map_err(SparkError::from)?;
        let Plan { op_type: op } = request.plan.required("plan")?;
        let op = op.required("plan op")?;
        let stream = match op {
            plan::OpType::Root(relation) => {
                service::handle_execute_relation(&ctx, relation, metadata).await?
            }
            plan::OpType::Command(Command { command_type: command }) => {
                let command = command.required("command")?;
                handle_command(&ctx, command, metadata).await?
            }
            plan::OpType::CompressedOperation(_) => {
                return Err(Status::unimplemented("compressed operation plan"));
            }
        };
        Ok(Response::new(stream))
    }
    // ...
}
```

The flow is: 1. Extract `session_id` and `user_id` from the request. 2. Call `get_or_create_session_context` — this either returns an existing DataFusion `SessionContext` or creates a fresh one, fully initialized with Spark semantics. 3. Inspect the plan's `op_type`: it is either a `Root` (a query/relation to execute) or a `Command` (a mutation like a write, DDL, or UDF registration). 4. Delegate to the appropriate handler in the `service` module. 5. Return a `Response::new(stream)` — the stream is `ExecutePlanResponseStream`, a wrapper around a tokio channel that produces `ExecutePlanResponse` values.

Routing Commands

The `handle_command` function routes protobuf `CommandType` variants to typed handlers:

```
async fn handle_command(
    ctx: &SessionContext,
    command: crate::spark::connect::command::CommandType,
    metadata: ExecutorMetadata,
) -> SparkResult<ExecutePlanResponseStream> {
    use crate::spark::connect::command::CommandType;

    match command {
        CommandType::RegisterFunction(udf) => {
            service::handle_execute_register_function(ctx, udf, metadata).await
```

```
        }
        CommandType::WriteOperation(write) => {
            service::handle_execute_write_operation(ctx, write, metadata).await
        }
        CommandType::CreateDataframeView(view) => {
            service::handle_execute_create_dataframe_view(ctx, view, metadata).await
        }
        CommandType::WriteOperationV2(write) => {
            service::handle_execute_write_operation_v2(ctx, write, metadata).await
        }
        CommandType::SqlCommand(sql) => {
            service::handle_execute_sql_command(ctx, sql, metadata).await
        }
        CommandType::MergeIntoTableCommand(command) => {
            service::handle_execute_merge_into_table_command(ctx, command,
metadata).await
        }
        CommandType::MlCommand(_) => Err(SparkError::todo("ml command")),
        // ... many more arms
    }
}
```

The pattern is consistent: each protobuf variant maps to a `service::handle_*` function that converts the protobuf type to the internal `spec` IR and calls into `sail-plan`. Unimplemented variants return `SparkError::todo(...)` which becomes a gRPC `UNIMPLEMENTED` status.

`AnalyzePlan`: Schema and Explain Without Executing

Not every Spark call triggers execution. `df.schema`, `df.explain()`, `df.isStreaming` — these call `AnalyzePlan`, a request/response RPC (no streaming). The server resolves the plan to a logical plan, inspects it, and returns the result without running the physical plan:

```
async fn analyze_plan(
    &self,
    request: Request<AnalyzePlanRequest>,
) -> Result<Response<AnalyzePlanResponse>, Status> {
    let analyze = request.analyze.required("analyze")?;
    let result = match analyze {
        Analyze::Schema(schema) => {
            let schema = service::handle_analyze_schema(&ctx, schema).await?;
            Some(analyze_plan_response::Result::Schema(schema))
        }
        Analyze::Explain(explain) => {
            let explain = service::handle_analyze_explain(&ctx, explain).await?;
            Some(analyze_plan_response::Result::Explain(explain))
        }
        Analyze::SparkVersion(version) => {
            let version = service::handle_analyze_spark_version(&ctx,
version).await?;
            Some(analyze_plan_response::Result::SparkVersion(version))
        }
        // ...
    };
    Ok(Response::new(AnalyzePlanResponse { result, .. }))
}
```

Session Management

Each PySpark client has a session identified by a UUID string and an optional user ID. Sail stores all per-session state — configuration, active executors, streaming queries — in a `SparkSession` struct that is embedded as an extension on DataFusion's `SessionContext`.

`SparkSession` as a `SessionExtension`

DataFusion's `SessionContext` has an extension map: `Arc<dyn Any + Send + Sync>`. Sail uses this to hang Spark-specific state off a DataFusion context without modifying DataFusion itself:

```
// crates/sail-spark-connect/src/session.rs
pub(crate) struct SparkSession {
    session_id: String,
    user_id: String,
    options: SparkSessionOptions,
    state: Mutex<SparkSessionState>,
}

impl SessionExtension for SparkSession {
    fn name() -> &'static str {
        "spark session"
    }
}
```

`SessionExtension` is a Sail trait (in `sail-common-datafusion`) that provides a typed `ctx.extension::<SparkSession>()` accessor. Concretely it downcasts from `Arc<dyn Any>` using the concrete type.

`SparkSessionState` contains: - `config: SparkRuntimeConfig` — a map of `spark.*` configuration keys and values - `executors: HashMap<String, Arc<Executor>>` — in-flight query operations, keyed by operation ID - `streaming_queries: StreamingQueryManager` — active streaming queries

Creating Sessions

Session creation happens in `session_manager.rs`. The interesting logic is in `SparkSessionMutator`, which intercepts the DataFusion `SessionContext` at construction time and adds the Spark extension:

```
impl ServerSessionMutator for SparkSessionMutator {
    fn mutate_config(
        &self,
        config: SessionConfig,
        info: &ServerSessionInfo,
    ) -> Result<SessionConfig> {
        let plan_service = PlanService::new(
            Box::new(DefaultCatalogDisplay::<SparkCatalogObjectDisplay>::default()),
            Box::new(SparkPlanFormatter),
        );
        let spark = SparkSession::try_new(
            info.session_id.clone(),
            info.user_id.clone(),
            SparkSessionOptions {
                execution_heartbeat_interval: Duration::from_secs(
                    self.config.spark.execution_heartbeat_interval_secs,
                ),
            },
        )?;
        Ok(config
            .with_extension(Arc::new(plan_service))
```

```
            .with_extension(Arc::new(spark)))
    }
}
```

Two extensions are added: a `PlanService` (which holds the `JobRunner`) and the `SparkSession`. Any code that has a `&SessionContext` can retrieve either of these with `ctx.extension::<SparkSession>()`.

The Executor: Buffering and Reattachment

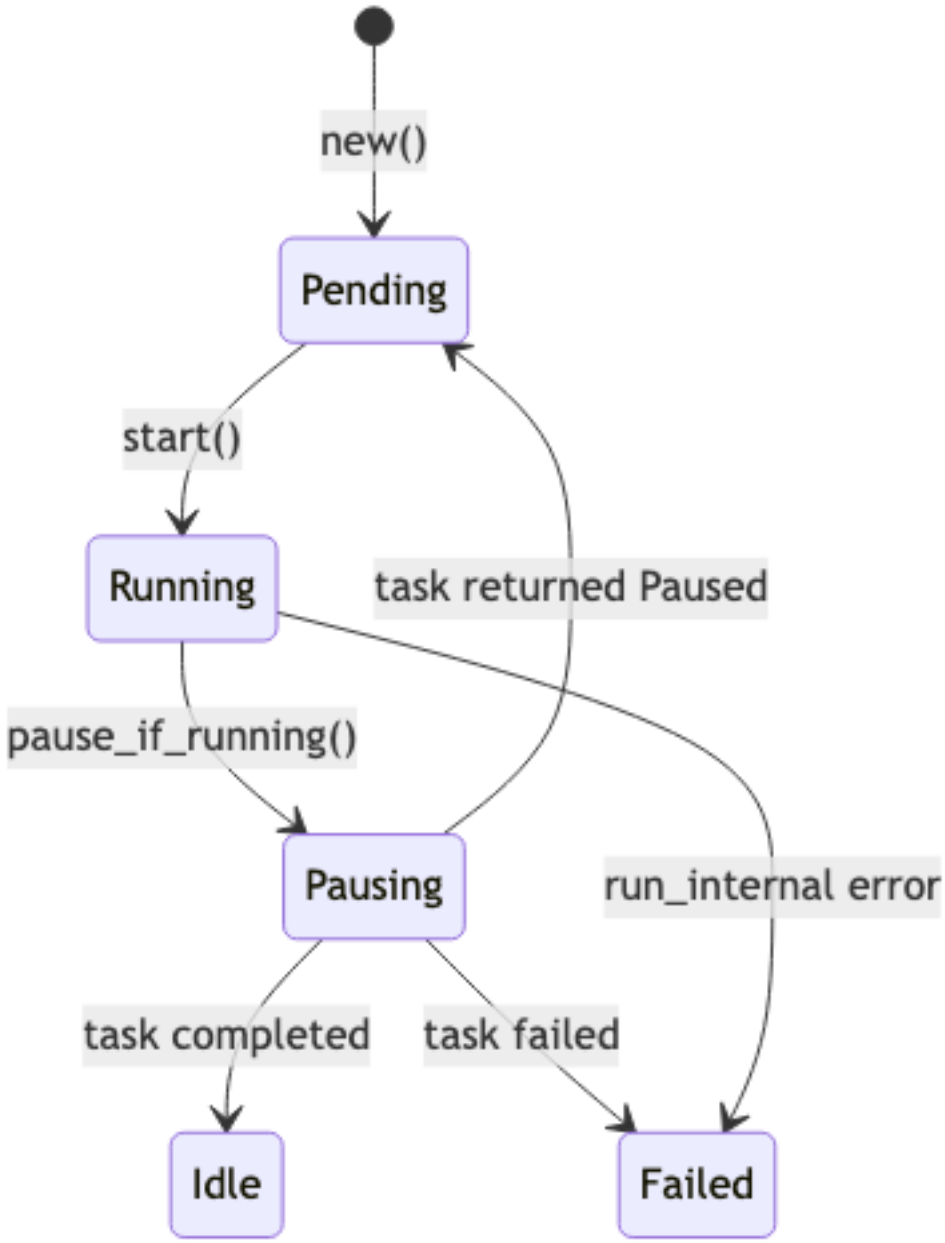
Spark Connect's reattachment feature requires the server to remember what it has already sent. When a client calls `ReattachExecute`, it passes a `last_response_id`; the server replays everything after that ID.

This is implemented in `crates/sail-spark-connect/src/executor.rs`. The `Executor` manages a tokio task that drains a `SendableRecordBatchStream` (DataFusion's streaming result type) and feeds a channel, while also recording every output in a ring buffer:

```
pub(crate) struct Executor {
    pub(crate) metadata: ExecutorMetadata,
    state: Mutex<ExecutorState>,
}

enum ExecutorState {
    Idle,
    Pending { context: ExecutorTaskContext, span: Span },
    Running { task: ExecutorTask, span: Span },
    Pausing,
    Failed(SparkError),
}
```

The state machine transitions:



The `run_internal` method serializes each `RecordBatch` to Arrow IPC format and sends it down the channel. Between batches it selects on a heartbeat timer, sending empty batches to keep the connection alive:

```
async fn run_internal(
    context: &mut ExecutorTaskContext,
    tx: mpsc::Sender<ExecutorOutput>,
) -> SparkResult<()> {
    // Replay any buffered outputs (for reattach)
    for out in context.replay_outputs()? {
        tx.send(out).await?;
    }
    // Send the schema first
    let schema = to_spark_schema(context.stream.schema())?;
    let out = ExecutorOutput::new(ExecutorBatch::Schema(Box::new(schema)));
    context.save_output(&out)?;
```

```
    tx.send(out).await?;

    let mut empty = true;
    while let Some(batch) = context.next().await? {
        let batch = to_arrow_batch(&batch)?;
        let out = ExecutorOutput::new(ExecutorBatch::ArrowBatch(batch));
        context.save_output(&out)?;
        tx.send(out).await?;
        empty = false;
    }
    // Send at least one empty batch for zero-row results
    if empty {
        let batch = RecordBatch::new_empty(context.stream.schema());
        let out =
ExecutorOutput::new(ExecutorBatch::ArrowBatch(to_arrow_batch(&batch)?));
        context.save_output(&out)?;
        tx.send(out).await?;
    }

    let out = ExecutorOutput::new(ExecutorBatch::Complete);
    context.save_output(&out)?;
    tx.send(out).await?;
    Ok(())
}
```

The `context.next()` method uses `tokio::select!` to either pull the next batch or emit a heartbeat after `heartbeat_interval`:

```
async fn next(&mut self) -> SparkResult<Option<RecordBatch>> {
    tokio::select! {
        batch = self.stream.next() => Ok(batch.transpose()?),
        _ = tokio::time::sleep(self.heartbeat_interval) => {
            Ok(Some(RecordBatch::new_empty(self.stream.schema())))
        }
    }
}
```

Serializing to Arrow IPC

The final step before data leaves the server is serialization. Arrow IPC (the "stream" format, not the "file" format) is what Spark Connect uses for `ArrowBatch` payloads. The conversion is in `executor.rs`:

```
pub(crate) fn to_arrow_batch(batch: &RecordBatch) -> SparkResult<ArrowBatch> {
    let mut output = ArrowBatch::default();
    {
        let cursor = Cursor::new(&mut output.data);
        let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
        writer.write(batch)?;
        output.row_count += batch.num_rows() as i64;
        writer.finish()?;
    }
    Ok(output)
}
```

`ArrowBatch` is a protobuf message with a `data: bytes` field and a `row_count: i64`. The `StreamWriter` writes the Arrow IPC stream format into a `Vec<u8>` via a `Cursor`. The protobuf is then embedded in the `ExecutePlanResponse` and sent over gRPC.

On the PySpark side, `pyspark-client` reads the `data` bytes with `pyarrow.ipc.open_stream(...)` and reconstructs the `RecordBatch`.

Config Management

PySpark frequently reads and writes Spark configuration keys (`spark.sql.shuffle.partitions`, etc.) via the `Config` RPC. Sail stores these per-session in `SparkRuntimeConfig`, a typed wrapper around a `HashMap<String, String>` with validation. The `config` RPC handler in `server.rs` dispatches to helpers like `handle_config_get`, `handle_config_set`, `handle_config_unset`, etc., all of which operate on the `SparkSession` embedded in the `SessionContext`.

The `service` Module

The actual conversion from protobuf to `spec::Plan` happens in `crates/sail-spark-connect/src/service/`. The module is organized into:

- `plan_executor.rs` — defines `ExecutePlanResponseStream` (the gRPC stream type) and the `handle_execute_relation` / `handle_execute_*` functions
- `plan_analyzer.rs` — `handle_analyze_*` functions for non-executing introspection
- `config_manager.rs` — config RPC handlers
- `artifact_manager.rs` — artifact upload (JARs, Python files)

The execute-relation path, for example, converts a protobuf `Relation` (the Spark Connect representation of a `DataFrame`) into a `spec::Relation`, wraps it in a `spec::Plan::Query`, then calls `resolve_and_execute_plan` from `sail-plan` to get back a physical plan and a `SendableRecordBatchStream`. This stream is handed to a new `Executor`, which is stored in the session and starts running.

Summary

`sail-spark-connect` translates the Spark Connect gRPC protocol into Rust. Its responsibilities are:

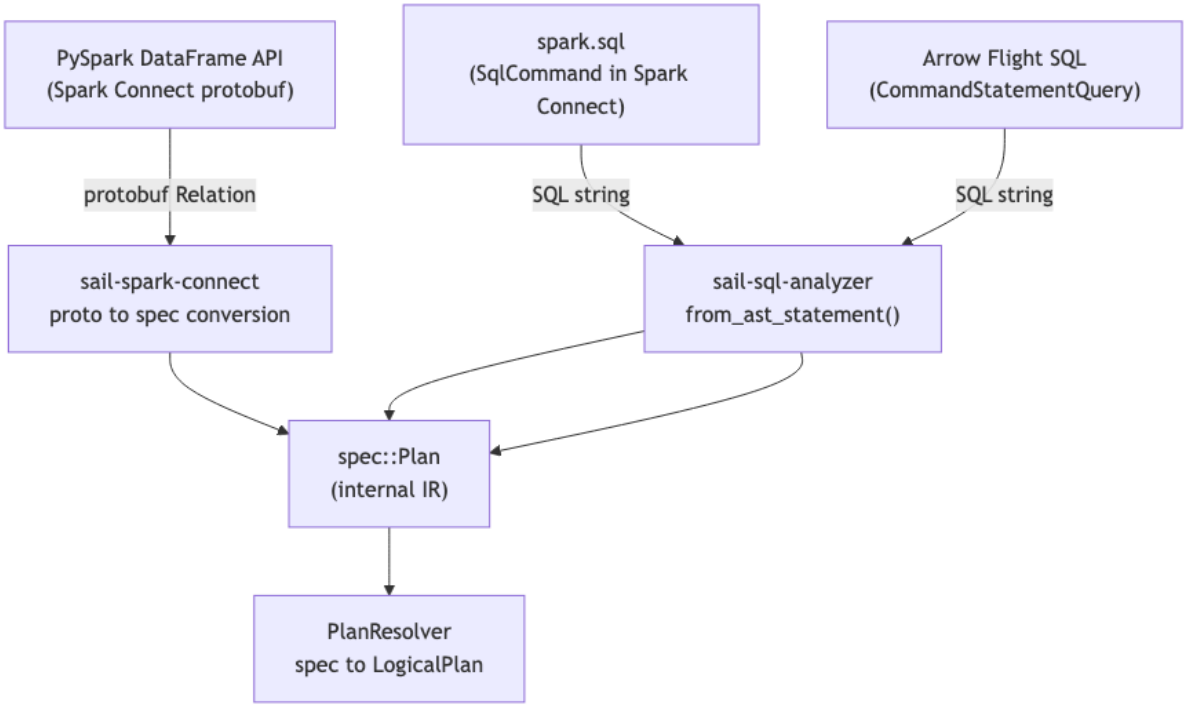
- Implementing the `SparkConnectService` trait with all nine RPC methods
- Managing sessions: creating, retrieving, and destroying DataFusion `SessionContext` instances
- Routing logical plans and commands to planning code in `sail-plan`
- Buffering executor output for reattachable streams
- Serializing Arrow `RecordBatch` values to IPC bytes for the wire

The crate knows nothing about how queries are planned or executed — that is the responsibility of `sail-plan` and `sail-execution`, described in the chapters that follow.

Chapter 2b: The SQL Pipeline

Three Roads to One IR

Sail has three entry points for queries. All of them converge on `spec::Plan`, the internal IR, before reaching the planning layer:



Chapter 2 covered the protobuf path. This chapter covers the SQL text path — what happens when `spark.sql("SELECT avg(amount) FROM orders WHERE dt > '2024-01'")` arrives.

`sail-sql-parser`: A Custom Parser

The SQL parser in Sail is entirely custom — not `sqlparser-rs`, not a fork of Calcite, not ANTLR. It is built with `chumsky` 0.12.0, a Rust parser combinator library with support for Pratt (top-down operator precedence) parsing.

The reason for a custom parser is full control over Spark SQL syntax. Spark SQL has significant divergences from standard SQL: `LATERAL VIEW`, `PIVOT`, `UNPIVOT`, `CLUSTER BY`, `DISTRIBUTE BY`, `SORT BY`, `REFRESH TABLE`, `ANALYZE TABLE`, `CACHE TABLE`, Hive compatibility syntax (`STORED AS`, `ROW FORMAT DELIMITED`, `SERDE`), and others. A general-purpose SQL parser would need extensive monkey-patching to handle all of these correctly; a bespoke parser is cleaner.

Keyword Codegen

The parser starts at build time. `crates/sail-sql-parser/build.rs` reads `data/keywords.txt` — a list of 368 SQL keywords in ASCII order:

```
# data/keywords.txt (excerpt)
ADD
AFTER
ALL
ALTER
ALWAYS
...
YEAR
YEARS
ZONE
```

The build script generates two Rust macros into `$OUT_DIR/keywords.rs`:

```
// Generated by build.rs
macro_rules! for_all_keywords {
```

```
    ($callback:ident) => { $callback!([("ADD", Add), ("AFTER", After), /* ... 368
total */]); }
}

macro_rules! keyword_map {
    ($value:ident) => { phf::phf_map! { "ADD" => $value!(Add), "AFTER" => $value!
(After), /* ... */ } }
}
```

Each keyword becomes a zero-sized Rust struct in `ast/keywords.rs` (e.g. `struct Add;`, `struct After;`). The `#[derive(TreeParser)]` proc-macro (described below) knows how to parse these — matching the keyword string against the lexed token stream. `phf::phf_map!` generates a perfect hash map for O(1) keyword lookup at parse time.

The Lexer

`crates/sail-sql-parser/src/lexer.rs` defines the token types and the lexer function. The lexer recognizes: - Keywords (looked up in the keyword perfect hash map) - Identifiers (unquoted and backtick-quoted) - String literals (single-quoted, with escape sequences and Unicode escape support: `U&"..."`) - Number literals (integer, decimal, hex) - Operators and punctuation - Comments (line and block, stripped from the token stream) - Dollar-sign parameters (`$1`, `?` for parameterized queries)

The lexer itself is a `chumsky` parser over `char` input that produces `Vec<(Token, Span)>`.

The `sail-sql-macro` Proc-Macros

`crates/sail-sql-macro/` defines three proc-macros that reduce the boilerplate of defining the grammar:

`#[derive(TreeParser)]` — generates a `chumsky` parser for the annotated type. For an enum, it generates a `choice(...)` over all variants. For a struct, it generates a sequential `then(...)` chain. The annotation attributes control dependencies (for recursive types) and custom parser functions:

```
// crates/sail-sql-parser/src/ast/query.rs
#[derive(Debug, Clone, TreeParser, TreeSyntax, TreeText)]
#[parser(dependency = "(Query, Expr, TableWithJoins)", label = TokenLabel::Query)]
pub struct Query {
    #[parser(function = |(q, _, _), o| compose(q, o))]
    pub with: Option<WithClause>,
    #[parser(function = |(q, e, t), o| boxed(compose((q, e, t), o)))]
    pub body: Box<QueryBody>,
    #[parser(function = |(_, e, _), o| compose(e, o))]
    pub modifiers: Vec<QueryModifier>,
}
```

`dependency = "(Query, Expr, TableWithJoins)"` means the generated `Query::parser()` method takes a tuple of parsers for these types as its argument — this is how `chumsky`'s `Recursive::declare()` / `Recursive::define()` cycle is handled without compiler errors on recursive types.

`#[derive(TreeSyntax)]` — generates a `syntax()` method that returns a human-readable grammar description of the type (used for error messages).

`#[derive(TreeText)]` — generates a `text()` method that unparses the AST back to normalized SQL text. The unparser is used in the gold tests to verify round-trip correctness.

Pratt Parsing for Expressions

SQL expressions have operator precedence — `a + b * c` must parse as `a + (b * c)`. chumsky's Pratt module handles this elegantly. The `Expr` type uses manual `impl TreeParser` rather than `#[derive(TreeParser)]` because its grammar has left recursion:

```
// crates/sail-sql-parser/src/ast/expression.rs
use chumsky::pratt::{infix, left, postfix, prefix, Operator};

// Expr implements TreeParser manually using chumsky's Pratt combinator
```

The Pratt parser defines operators with explicit precedences and associativities (`infix(left(...))`, `prefix(...)`, `postfix(...)`), producing the correct parse tree for complex expressions including: `IS NULL`, `IS NOT DISTINCT FROM`, `BETWEEN`, `LIKE`, `ILIKE`, `RLIKE`, `IN`, window functions, cast (`::` shorthand), subscript (`[...]`), and field access (`.`).

The Recursive Parser Structure

The top-level parser in `crates/sail-sql-parser/src/parser.rs` manually declares and defines recursive parsers for mutually-recursive types:

```
fn statement<'a, I, E>(options: &'a ParserOptions) -> impl Parser<'a, I, Statement,
E> + Clone {
    let mut statement  = Recursive::declare();
    let mut query      = Recursive::declare();
    let mut expression = Recursive::declare();
    let mut data_type  = Recursive::declare();
    let mut table_with_joins = Recursive::declare();

    statement.define(Statement::parser(
        (statement.clone(), query.clone(), expression.clone(), data_type.clone()),
        options,
    ));
    query.define(Query::parser(
        (query.clone(), expression.clone(), table_with_joins.clone()),
        options,
    ));
    expression.define(Expr::parser(
        (expression.clone(), query.clone(), data_type.clone()),
        options,
    ));
    // ...
    statement
}
```

`Recursive::declare()` creates a parser placeholder; `define()` fills it in. This two-step process allows the parser to refer to itself without triggering infinite loops at construction time.

Public API

`crates/sail-sql-analyzer/src/parser.rs` re-exports the parsing entry points:

```
// crates/sail-sql-analyzer/src/parser.rs
pub fn parse_one_statement(s: &str) -> SqlResult<Statement> { /* ... */ }
pub fn parse_statements(s: &str) -> SqlResult<Vec<Statement>> { /* ... */ }
pub fn parse_expression(s: &str) -> SqlResult<Expr> { /* ... */ }
pub fn parse_data_type(s: &str) -> SqlResult<DataType> { /* ... */ }
pub fn parse_interval(s: &str) -> SqlResult<IntervalValue> { /* ... */ }
pub fn parse_date(s: &str) -> SqlResult<DateValue> { /* ... */ }
pub fn parse_timestamp(s: &str) -> SqlResult<TimestampValue<'_>> { /* ... */ }
```

These are used throughout Sail: `parse_one_statement` in `sail-flight`'s `get_flight_info_statement`, `parse_data_type` in the plan resolver for DDL statements, `parse_date`/`parse_timestamp` in literal expression resolution.

`sail-sql-analyzer`: AST → `spec::Plan`

`sail-sql-analyzer` converts the `sail-sql-parser` AST into `sail-common`'s `spec::Plan`. This is where semantic structure is made explicit: a flat list of tokens becomes a typed IR node.

Statement Conversion

The entry point is `from_ast_statement` in `crates/sail-sql-analyzer/src/statement.rs`:

```
pub fn from_ast_statement(statement: Statement) -> SqlResult<spec::Plan> {
    match statement {
        Statement::Query(query) => {
            let plan = from_ast_query(query)?;
            Ok(spec::Plan::Query(plan))
        }
        Statement::SetCatalog { name, .. } => {
            Ok(spec::Plan::Command(spec::CommandPlan::new(
                spec::CommandNode::SetCurrentCatalog { catalog: name.into() }
            )))
        }
        Statement::CreateDatabase { name, if_not_exists, clauses, .. } => {
            let CreateDatabaseClauses { comment, location, properties } =
clauses.try_into()?;
            Ok(spec::Plan::Command(spec::CommandPlan::new(
                spec::CommandNode::CreateDatabase {
                    database: from_ast_object_name(name)?,
                    definition: spec::DatabaseDefinition {
                        if_not_exists: if_not_exists.is_some(),
                        comment: comment.map(from_ast_string).transpose()?,
                        location: location.map(from_ast_string).transpose()?,
                        properties: /* ... */,
                    },
                }
            )))
        }
        Statement::AlterDatabase { .. } => Err(SqlError::todo("ALTER DATABASE")),
        // ... 50+ more arms
    }
}
```

Each `Statement` variant maps to a `spec::Plan::Command(CommandPlan { node: CommandNode::... })` or a `spec::Plan::Query(QueryPlan { node: QueryNode::... })`. Unimplemented paths return `SqlError::todo(...)`.

Data Type Conversion

`from_ast_data_type` converts the parser's `DataType` AST node into `spec::DataType`. The parser supports all Spark SQL and ANSI SQL type aliases:

```
// crates/sail-sql-analyzer/src/data_type.rs
pub fn from_ast_data_type(sql_type: DataType) -> SqlResult<spec::DataType> {
    match sql_type {
        DataType::Null(_) | DataType::Void(_) => Ok(spec::DataType::Null),
        DataType::Boolean(_) | DataType::Bool(_) => Ok(spec::DataType::Boolean),
        DataType::TinyInt(None, _) | DataType::Byte(None, _) | DataType::Int8(_) => {
```

```
            Ok(spec::DataType::Int8)
        }
        DataType::BigInt(None, _) | DataType::Long(None, _) | DataType::Int64(_) => {
            Ok(spec::DataType::Int64)
        }
        DataType::TinyInt(Some(_), _) => Ok(spec::DataType::UInt8),  // UNSIGNED
modifier
        DataType::Decimal(_, info) => {
            let (precision, scale) = /* parse from AST */ ?;
            Ok(spec::DataType::Decimal128 { precision, scale })
        }
        // ... handles UNSIGNED integers, CHAR, VARCHAR, BINARY,
        //     TIMESTAMP, TIMESTAMP_LTZ, TIMESTAMP_NTZ, DATE, TIME,
        //     INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND,
        //     ARRAY<T>, MAP<K,V>, STRUCT<f: T>, ...
    }
}
```

`DataType::TinyInt(Some(_))` matches `TINYINT UNSIGNED` — the `Some(_)` captures the `UNSIGNED` keyword. This is an example of the parser preserving syntax details (unsigned modifier) that the analyzer uses for semantic mapping.

Expression Conversion

`from_ast_expression` in `crates/sail-sql-analyzer/src/expression.rs` handles the deeply nested `Expr` AST. It recursively converts each expression variant into a `spec::Expr`. For example, window functions:

```
Expr::WindowFunction(func, over) => {
    // func is a FunctionExpr (name + args)
    // over is an OverClause (PARTITION BY, ORDER BY, WINDOW FRAME)
    let window_spec = from_ast_window_spec(over.spec)?;
    let function = from_ast_function_expr(func)?;
    Ok(spec::Expr::Window { function: Box::new(function), window_spec })
}
```

Lambda expressions (for `TRANSFORM`, `FILTER`, `AGGREGATE`) are handled specially because they introduce named variables that are not column references:

```
Expr::Lambda(params, arrow, body) => {
    let variables = from_ast_lambda_params(params)?;
    let function = Box::new(from_ast_expression(*body)?);
    Ok(spec::Expr::Lambda { function, arguments: variables })
}
```

Interval, Date, and Timestamp Parsing

Interval literals in Spark SQL have complex syntax: `INTERVAL '1-3' YEAR TO MONTH`, `INTERVAL 5 DAYS`, `INTERVAL '01:30:00' HOUR TO SECOND`. The analyzer has dedicated parsers for these in `crates/sail-sql-analyzer/src/literal/`:

```
// crates/sail-sql-analyzer/src/parser.rs
pub fn parse_interval(s: &str) -> SqlResult<IntervalValue> {
    parse_unqualified_interval_string(s, false)
}
pub fn parse_date(s: &str) -> SqlResult<DateValue> {
    parse_simple!(s, create_date_parser)
}
pub fn parse_timestamp(s: &str) -> SqlResult<TimestampValue<'_>> {
```

```
    parse_simple!(s, create_timestamp_parser)
}
```

These standalone parsers are used by the plan resolver when it encounters date/timestamp literals in expressions, ensuring Spark-compatible parsing of formats like `'2024-01-15'` and `'2024-01-15 10:30:00.123'`.

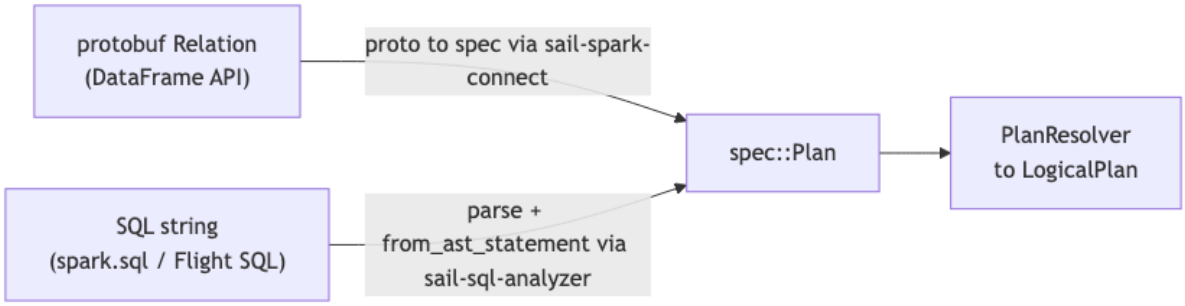
The SqlCommand Path

When a Spark Connect client sends `spark.sql("SELECT ...")`, it arrives as a `CommandType::SqlCommand` in `execute_plan`. The handler in `sail-spark-connect/src/service/plan_executor.rs`:

```
CommandType::SqlCommand(sql) => {
    service::handle_execute_sql_command(ctx, sql, metadata).await
}
```

`handle_execute_sql_command` extracts the SQL string, calls `parse_one_statement`, then `from_ast_statement`, producing a `spec::Plan` that goes through the same `resolve_and_execute_plan` pipeline as a protobuf-originated plan. The two paths are completely symmetric from the resolver's perspective.

The Three Paths Converge



`PlanResolver` in `sail-plan` consumes `spec::Plan` exclusively. It has no knowledge of whether the plan came from a Spark Connect protobuf or a SQL string — the IR is the same. This is the key design property that allows Sail to add new entry points (e.g. a future REST API, or the existing Flight SQL endpoint) without touching the planning layer.

Gold Tests and the Parser

`sail-gold-test` uses `parse_one_statement` + `from_ast_statement` to replay Spark's function documentation examples as SQL queries. Each function's docstring examples become test cases: parse the SQL, execute against Sail, compare output against Spark's expected output. This is how Sail verifies SQL-level compatibility systematically.

The parser's `TreeText` derive also enables a round-trip check: `parse_one_statement(sql)?.text()` should be equivalent (modulo whitespace) to the original SQL. The test suite in `sail-sql-analyzer` verifies this:

```
#[test]
fn test_unparse() -> SqlResult<()> {
    assert_eq!(
        parse_one_statement("/* */ SELECT 1+1")?.text(),
        "SELECT 1 + 1 "
    );
    assert_eq!(
```

```
        parse_one_statement("SELECT foo(0), cast(1L as decimal(10, -1)) FROM
a.b")?.text(),
        "SELECT foo ( 0 ) , CAST ( 1L AS DECIMAL ( 10 , -1 ) ) FROM a . b "
    );
    Ok(())
}
```

Summary

`sail-sql-parser` is a complete, from-scratch SQL parser using `chumsky`: - 368 keywords generated at build time into a perfect hash map - Proc-macros derive recursive-descent parsers from annotated AST structs - Pratt parsing handles operator precedence in expressions - The AST preserves full syntactic detail (keyword positions, whitespace spans) for unparser fidelity

`sail-sql-analyzer` converts the AST to `spec::Plan` with full Spark semantic mapping — handling UNSIGNED types, interval subtypes, lambda parameters, and the full DDL/DML statement set. Together they form the SQL entry path that parallels the protobuf path, converging at `spec::Plan` before reaching `PlanResolver`.

Chapter 3: Apache Arrow — Sail's Data Backbone

What Is Apache Arrow?

Apache Arrow is a specification for a columnar, language-agnostic, zero-copy in-memory data format, together with a growing set of implementations in C++, Rust, Python, Java, Go, and others. The core idea is simple but powerful: data is stored column-by-column rather than row-by-row, and the in-memory layout is standardized so that two processes that both use Arrow can share data by passing a pointer — no serialization, no copying.

For a query engine, this matters a great deal. Most analytical operations — aggregations, filters, projections — access all the values in a column before moving to the next column. Columnar layout means those values are contiguous in memory: the CPU prefetcher works, SIMD instructions apply cleanly, and zero-copy batch hand-offs between operators are possible.

Sail uses the `arrow` Rust crate (part of the Arrow project's native Rust implementation, also used by DataFusion). The two foundational types are `Schema` and `RecordBatch`.

`Schema` and `Field`

An Arrow `Schema` describes the column layout of a batch of data: it is a list of `Field` values, each carrying a name, a `DataType`, and a nullable flag, plus optional key-value metadata.

```
use datafusion::arrow::datatypes::{DataType, Field, Schema};
use std::sync::Arc;

let schema = Schema::new(vec![
    Field::new("id",     DataType::Int64,  false),
    Field::new("amount", DataType::Float64, true),
    Field::new("name",   DataType::Utf8,   true),
]);
let schema_ref: Arc<Schema> = Arc::new(schema);
```

`Arc<Schema>` (aliased as `SchemaRef`) is ubiquitous in Sail and DataFusion. Schemas are immutable and shared — multiple `RecordBatches` from the same scan share one `Arc<Schema>`.

Arrow's `DataType` is a rich enum:

```
pub enum DataType {
    Null, Boolean,
    Int8, Int16, Int32, Int64,
    UInt8, UInt16, UInt32, UInt64,
    Float16, Float32, Float64,
    Timestamp(TimeUnit, Option<Arc<str>>),  // timezone
    Date32, Date64,
    Time32(TimeUnit), Time64(TimeUnit),
    Duration(TimeUnit),
    Interval(IntervalUnit),
    Binary, LargeBinary, FixedSizeBinary(i32),
    Utf8, LargeUtf8,
    List(Arc<Field>), LargeList(Arc<Field>), FixedSizeList(Arc<Field>, i32),
    Struct(Fields),
    Map(Arc<Field>, bool),   // (entries field, keys_sorted)
    Decimal128(u8, i8), Decimal256(u8, i8),
    // ... extension types, dictionaries, unions
}
```

The nested types (`List`, `Struct`, `Map`) contain child `Field` definitions, allowing arbitrary nesting — Spark's nested DataFrames map directly.

`RecordBatch`

A `RecordBatch` is a table: a schema plus one `Arc<dyn Array>` per column, all with the same number of rows. The `dyn Array` abstraction is how Arrow handles type-heterogeneous columns: at runtime each column is a concrete typed array (e.g. `Int64Array`, `StringArray`, `StructArray`), all sharing the common `Array` interface.

```
use datafusion::arrow::array::{Int64Array, StringArray};
use datafusion::arrow::record_batch::RecordBatch;

let ids    = Arc::new(Int64Array::from(vec![1, 2, 3]));
let names  = Arc::new(StringArray::from(vec!["alice", "bob", "carol"]));
let batch  = RecordBatch::try_new(Arc::new(schema), vec![ids, names]).unwrap();

println!("{} rows, {} columns", batch.num_rows(), batch.num_columns());
```

`RecordBatch` is the unit of work throughout Sail. Every `ExecutionPlan::execute` returns a `SendableRecordBatchStream`:

```
type SendableRecordBatchStream = Pin<Box<dyn RecordBatchStream + Send>>;
```

where `RecordBatchStream` is:

```
pub trait RecordBatchStream: Stream<Item = Result<RecordBatch>> {
    fn schema(&self) -> SchemaRef;
}
```

Operators pull batches from their input streams, process them, and emit batches downstream. When a batch reaches the top of the physical plan tree, it is in the `Executor` in `sail-spark-connect`, waiting to be serialized to Arrow IPC.

Spark Types → Arrow Types: The Mapping

Spark has its own type system. The Spark Connect protobuf defines types like `ByteType`, `ShortType`, `LongType`, `TimestampType`, `TimestampNtzType`, `DayTimeIntervalType`, and so on. Sail converts these through two stages:

1. **Proto → `spec::DataType`**: `sail-spark-connect`'s proto conversion layer maps protobuf `data_type::Kind` variants to `sail-common`'s `spec::DataType` enum.
2. **`spec::DataType` → `adt::DataType`**: `PlanResolver::resolve_data_type` in `crates/sail-plan/src/resolver/data_type.rs` maps the internal IR to Arrow's type system.

The second mapping is the interesting one because it has subtleties the proto-to-spec stage cannot resolve alone:

```
// crates/sail-plan/src/resolver/data_type.rs
pub(super) fn resolve_data_type(
    &self,
    data_type: &spec::DataType,
    state: &mut PlanResolverState,
) -> PlanResult<adt::DataType> {
    use spec::DataType;

    match data_type {
        DataType::Null    => Ok(adt::DataType::Null),
        DataType::Boolean => Ok(adt::DataType::Boolean),
        DataType::Int8    => Ok(adt::DataType::Int8),
        DataType::Int16   => Ok(adt::DataType::Int16),
        DataType::Int32   => Ok(adt::DataType::Int32),
        DataType::Int64   => Ok(adt::DataType::Int64),
        DataType::Float32 => Ok(adt::DataType::Float32),
        DataType::Float64 => Ok(adt::DataType::Float64),
        DataType::Timestamp { time_unit, timestamp_type } =>
Ok(adt::DataType::Timestamp(
            Self::resolve_time_unit(time_unit)?,
            self.resolve_timezone(timestamp_type)?,
        )),
        DataType::List { element_type, element_nullable, metadata } => {
            Ok(adt::DataType::List(Arc::new(
                self.resolve_field_with_metadata(
                    SAIL_LIST_FIELD_NAME,
                    element_type,
                    *element_nullable,
                    metadata.iter().cloned(),
                    state,
                )?,
            )))
        }
        DataType::Struct { fields } => {
            Ok(adt::DataType::Struct(self.resolve_fields(fields, state)?))
        }
        DataType::Decimal128 { precision, scale } => {
            Ok(adt::DataType::Decimal128(*precision, *scale))
        }
        DataType::Map { key_type, value_type, value_nullable, .. } => {
            // Map in Arrow is: List<entries: Struct<key: K, value: V>>
            Ok(adt::DataType::Map(Arc::new(/* ... */), false))
        }
        // ...
    }
}
```

The timestamp subtlety. Spark's `TimestampType` is "timestamp with local time zone" — values are stored in UTC but displayed in the session timezone. Spark's `TimestampNtzType` has no timezone. In

Arrow, `Timestamp(Microseconds, Some("UTC"))` represents the former and `Timestamp(Microseconds, None)` the latter. This distinction is handled by `resolve_timezone`:

```
fn resolve_timezone(
    &self,
    timestamp_type: &spec::TimestampType,
) -> PlanResult<Option<Arc<str>>> {
    match timestamp_type {
        spec::TimestampType::Configured => match self.config.default_timestamp_type {
            DefaultTimestampType::TimestampLtz => Ok(Some(Arc::from("UTC"))),
            DefaultTimestampType::TimestampNtz => Ok(None),
        },
        spec::TimestampType::WithLocalTimeZone  => Ok(Some(Arc::from("UTC"))),
        spec::TimestampType::WithoutTimeZone    => Ok(None),
    }
}
```

The `default_timestamp_type` configuration key (`spark.sql.timestampType`) lets users switch the behavior globally — Sail tracks this per session via the `SparkRuntimeConfig`.

The string/binary subtlety. Arrow distinguishes between `Utf8` (32-bit offsets, ≤2 GiB per column) and `LargeUtf8` (64-bit offsets). Most tools produce `Utf8` by default. Sail has a configuration flag `arrow_use_large_var_types` that, when set, uses `LargeUtf8` and `LargeBinary` instead — useful for very wide string columns:

```
fn arrow_string_type(&self, state: &mut PlanResolverState) -> adt::DataType {
    if self.config.arrow_use_large_var_types
        && state.config().arrow_allow_large_var_types
    {
        adt::DataType::LargeUtf8
    } else {
        adt::DataType::Utf8
    }
}
```

Schema → Spark Schema: The Reverse Direction

When Sail needs to send a schema back to PySpark (e.g. in response to `df.schema`), it converts an Arrow `SchemaRef` into a Spark Connect `DataType` (specifically a `DataType::Struct`). This happens in `crates/sail-spark-connect/src/schema.rs`:

```
// crates/sail-spark-connect/src/schema.rs
pub(crate) fn to_spark_schema(schema: SchemaRef) -> SparkResult<sc::DataType> {
    DataType::Struct(schema.fields().clone()).try_into()
}
```

The `TryInto` implementation performs the inverse mapping: Arrow `DataType` → protobuf `sc::DataType`. This conversion also needs to handle extension types (like GeoArrow geometry columns) and Arrow metadata (field-level key-value pairs that encode extra Spark semantics).

Arrow IPC: Streaming Format

The on-wire format between Sail and PySpark is Arrow IPC, specifically the *streaming* format (as opposed to the *file* format which has a footer). The streaming format is a sequence of:

```
[schema message]
[record batch message]*
[end-of-stream marker]
```

Each message is a byte sequence: a 4-byte length prefix, a FlatBuffers-encoded header describing the buffer layout, then the raw buffer data. Sail writes this with `arrow::ipc::writer::StreamWriter`:

```
// crates/sail-spark-connect/src/executor.rs
pub(crate) fn to_arrow_batch(batch: &RecordBatch) -> SparkResult<ArrowBatch> {
    let mut output = ArrowBatch::default();
    {
        let cursor = Cursor::new(&mut output.data);
        let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
        writer.write(batch)?;
        output.row_count += batch.num_rows() as i64;
        writer.finish()?;
    }
    Ok(output)
}
```

Each `ArrowBatch` protobuf contains: - `data: bytes` — the complete IPC stream (schema + one batch) - `row_count: i64` — the number of rows, used by PySpark to allocate buffers

On the Python side, `pyarrow.ipc.open_stream(pa.py_buffer(data)).read_all()` reconstructs the `RecordBatch` from bytes. Because both sides use the same IPC format definition, there is no custom serialization layer — Arrow is the protocol.

Arrow in the Execution Layer

Arrow's zero-copy architecture influences Sail's execution operators directly. The `RowRoundRobinPartitioner` in `crates/sail-physical-plan/src/repartition.rs` redistributes rows across output partitions without copying column buffers unnecessarily:

```
pub fn partition<F>(&mut self, batch: RecordBatch, mut f: F) -> Result<()>
where
    F: FnMut(usize, RecordBatch) -> Result<()>,
{
    let schema = batch.schema();
    let mut indices = vec![Vec::new(); self.num_partitions];
    for row_index in 0..batch.num_rows() {
        let partition = (self.next_idx + row_index) % self.num_partitions;
        indices[partition].push(row_index as u32);
    }
    self.next_idx = (self.next_idx + batch.num_rows()) % self.num_partitions;

    for (partition, partition_indices) in indices.into_iter().enumerate() {
        if partition_indices.is_empty() { continue; }
        let indices_array: PrimitiveArray<UInt32Type> = partition_indices.into();
        let columns = take_arrays(batch.columns(), &indices_array, None)?;
        let options = RecordBatchOptions::new()
            .with_row_count(Some(indices_array.len()));
        let partition_batch =
            RecordBatch::try_new_with_options(schema.clone(), columns, &options)?;
        f(partition, partition_batch)?;
    }
    Ok(())
}
```

`take_arrays` is Arrow's gather operation: given an array and an array of indices, it produces a new array by selecting the rows at those indices. This is a fundamental Arrow operation that the C++ and Rust implementations have highly optimized, including SIMD paths for fixed-width types.

Summary

Arrow is not just a wire format for Sail — it is the data model at every level:

- **Planning**: schemas and field definitions drive type inference and validation.
- **Execution**: `RecordBatch` is the unit passed between operators; Arrow compute kernels do the actual work.
- **Transport**: Arrow IPC streams encode results for the Spark Connect wire.

The type mapping between Spark's type system and Arrow's type system is handled in `sail-plan/src/resolver/data_type.rs` with careful attention to Spark-specific subtleties — especially around timestamps, nullable semantics, and large variable-length types.

Chapter 4: Apache DataFusion — The Query Engine Core

Why DataFusion?

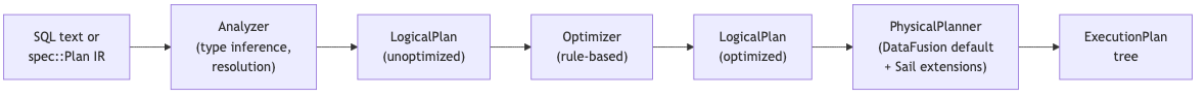
DataFusion is an in-process, extensible query engine written in Rust. It provides: a SQL parser, an expression type system, a rule-based logical optimizer, a physical planner, and a streaming execution runtime built on Arrow and tokio. It is the foundation that Sail's query planning is built on.

The choice of DataFusion is important to understand. Sail does not re-implement query planning from scratch. Instead, it extends DataFusion in well-defined ways: - Custom `UserDefinedLogicalNodeCore` nodes for Spark-specific logical plan constructs - Custom `ExecutionPlan` implementations for Spark-specific physical operators - Custom optimizer rules for Spark-specific rewrites - Custom catalog implementations behind DataFusion's catalog trait - A session extension mechanism to layer Spark state onto DataFusion contexts

This architecture means that everything DataFusion does well — predicate pushdown, projection pruning, join reordering, aggregate partial computation, Parquet column pruning — Sail gets for free. Sail focuses its engineering on the Spark-specific layer.

The Planning Pipeline

DataFusion's planning pipeline has three stages:



Sail adds a fourth stage before SQL parsing — the conversion from Spark Connect's protobuf through `spec::Plan` to DataFusion's `LogicalPlan`. This is `PlanResolver`.

`resolve_and_execute_plan`

The central function that drives the full pipeline is in `crates/sail-plan/src/lib.rs`:

```
pub async fn resolve_and_execute_plan(
    ctx: &SessionContext,
    config: Arc<PlanConfig>,
    plan: spec::Plan,
) -> PlanResult<(Arc<dyn ExecutionPlan>, Vec<StringifiedPlan>)> {
    let mut info = vec![];
    let resolver = PlanResolver::new(ctx, config);
    let NamedPlan { plan, fields } = resolver.resolve_named_plan(plan).await?;
    info.push(plan.to_stringified(PlanType::InitialLogicalPlan));

    let df = execute_logical_plan(ctx, plan).await?;
    let (session_state, plan) = df.into_parts();
```

```
    let plan = session_state.optimize(&plan)?;

    let plan = if is_streaming_plan(&plan)? {
        rewrite_streaming_plan(plan)?
    } else {
        plan
    };
    info.push(plan.to_stringified(PlanType::FinalLogicalPlan));

    let plan = session_state
        .query_planner()
        .create_physical_plan(&plan, &session_state)
        .await?;
    let plan = if let Some(fields) = fields {
        rename_physical_plan(plan, &fields)?
    } else {
        plan
    };
    info.push(StringifiedPlan::new(
        PlanType::FinalPhysicalPlan,
        displayable(plan.as_ref()).indent(true).to_string(),
    ));
    Ok((plan, info))
}
```

The steps: 1. **Resolve**: `PlanResolver::resolve_named_plan` converts `spec::Plan` → DataFusion `LogicalPlan`. 2. **Execute-to-DataFrame**: `ctx.execute_logical_plan(plan)` — this is DataFusion's entry point; it runs the analyzer. 3. **Optimize**: `session_state.optimize(&plan)` — runs the optimizer rule chain. 4. **Streaming check**: if the plan is a streaming plan, rewrite it for the streaming physical executor. 5. **Physical plan**: `session_state.query_planner().create_physical_plan(...)` — runs the physical planner. 6. **Rename**: if the query had user-facing column aliases, apply them to the physical plan's schema.

The `Vec<StringifiedPlan>` return carries explain-plan strings for each stage: initial logical, final logical, and final physical. These are used by `df.explain()`.

`PlanResolver`: spec::Plan → LogicalPlan

`PlanResolver` in `crates/sail-plan/src/resolver/` is the largest single piece of planning code. It converts every Spark Connect relation type and expression type into DataFusion's equivalents.

The entry point:

```
// crates/sail-plan/src/resolver/plan.rs
impl PlanResolver<'_> {
    pub async fn resolve_named_plan(&self, plan: spec::Plan) -> PlanResult<NamedPlan>
{
        let mut state = PlanResolverState::new();
        match plan {
            spec::Plan::Query(query) => {
                let plan = self.resolve_query_plan(query, &mut state).await?;
                let fields = Some(Self::get_field_names(plan.schema(), &state)?);
                Ok(NamedPlan { plan, fields })
            }
            spec::Plan::Command(command) => {
                let plan = self.resolve_command_plan(command, &mut state).await?;
                Ok(NamedPlan { plan, fields: None })
```

```
            }
        }
    }
}
```

`PlanResolverState` carries resolution state that is accumulated during traversal: field aliases, column renaming maps, and context flags. It is threaded through the recursive resolver methods.

The resolver handles dozens of relation types. For example, a `Filter` relation (corresponding to `df.filter(...)`) is resolved by recursively resolving the input relation and the filter expression:

```
spec::Relation::Filter(filter) => {
    let input = self.resolve_query_plan(*filter.input, state).await?;
    let predicate = self.resolve_expression(&filter.condition, input.schema(),
state)?;
    Ok(LogicalPlan::Filter(Filter::try_new(predicate, Arc::new(input))?))
}
```

Custom Logical Plan Nodes

Spark has concepts that do not map cleanly to DataFusion's built-in `LogicalPlan` variants. Sail adds them as `UserDefinedLogicalNodeCore` implementations in `crates/sail-logical-plan/`.

`RangeNode`: Spark's `spark.range()`

`spark.range(start, end, step, numPartitions)` generates a sequence of integers. DataFusion has no built-in for this. Sail implements it as a leaf logical node:

```
// crates/sail-logical-plan/src/range.rs
#[derive(Clone, Debug, PartialEq, Eq, Hash, Educe)]
#[educe(PartialOrd)]
pub struct RangeNode {
    range: Range,
    num_partitions: usize,
    #[educe(PartialOrd(ignore))]
    schema: DFSchemaRef,
}

impl UserDefinedLogicalNodeCore for RangeNode {
    fn name(&self) -> &str { "Range" }

    fn inputs(&self) -> Vec<&LogicalPlan> { vec![] }  // leaf node

    fn schema(&self) -> &DFSchemaRef { &self.schema }

    fn expressions(&self) -> Vec<Expr> { vec![] }

    fn fmt_for_explain(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "Range: start={}, end={}, step={}, num_partitions={}",
            self.range.start, self.range.end, self.range.step, self.num_partitions)
    }

    fn with_exprs_and_inputs(&self, exprs: Vec<Expr>, inputs: Vec<LogicalPlan>) ->
Result<Self> {
        exprs.zero()?;
        inputs.zero()?;
        Ok(self.clone())
    }
}
```

The `Range` struct also implements partitioning logic for splitting the range across workers:

```
impl Range {
    pub fn partition(&self, partition: usize, num_partitions: usize) -> Self {
        let start = self.start as i128;
        let end   = self.end   as i128;
        let step  = self.step  as i128;
        let num_elements = /* ... element count, accounting for direction */;
        let num_partitions = num_partitions as i128;
        let partition      = partition      as i128;
        let partition_start = partition       * num_elements / num_partitions * step
+ start;
        let partition_end   = (partition + 1) * num_elements / num_partitions * step
+ start;
        Range { start: partition_start as i64, end: partition_end as i64, step: step
as i64 }
    }
}
```

`ExplicitRepartitionNode`: Coalesce, RoundRobin, Hash

Spark's `coalesce()`, `repartition()`, and `repartitionByRange()` have specific semantics. Sail models these as a single logical node with a `kind` discriminant:

```
// crates/sail-logical-plan/src/repartition.rs
#[derive(Clone, Copy, Debug, Eq, PartialEq, Hash, PartialOrd, Ord)]
pub enum ExplicitRepartitionKind {
    Coalesce,
    RoundRobin,
    Hash,
}

#[derive(Clone, Debug, Eq, PartialEq, Hash, PartialOrd)]
pub struct ExplicitRepartitionNode {
    input: Arc<LogicalPlan>,
    num_partitions: Option<usize>,
    kind: ExplicitRepartitionKind,
    partitioning_expressions: Vec<Expr>,
}

impl UserDefinedLogicalNodeCore for ExplicitRepartitionNode {
    fn name(&self) -> &str { "ExplicitRepartition" }

    fn inputs(&self) -> Vec<&LogicalPlan> { vec![&self.input] }

    fn schema(&self) -> &DFSchemaRef { self.input.schema() }

    fn necessary_children_exprs(&self, output_columns: &[usize]) ->
Option<Vec<Vec<usize>>> {
        Some(vec![output_columns.to_vec()])
    }

    fn with_exprs_and_inputs(&self, exprs: Vec<Expr>, mut inputs: Vec<LogicalPlan>) -
> Result<Self> {
        let (Some(input), true) = (inputs.pop(), inputs.is_empty()) else {
            return plan_err!("{} expects exactly one input", self.name());
        };
        Ok(Self::new(Arc::new(input), self.num_partitions, self.kind, exprs))
```

```
    }
}
```

The `necessary_children_exprs` method tells the optimizer which output columns are needed from the child, enabling projection pushdown even through repartition nodes.

The Logical Optimizer

DataFusion's logical optimizer runs a rule chain over the `LogicalPlan` tree. Sail has two distinct optimizer layers: *logical* (in `sail-logical-optimizer`) and *physical* (in `sail-physical-optimizer`). They are separate pipelines with different insertion points.

`DecorrelateLateralProjection` (Logical)

Spark supports `LATERAL` subqueries. DataFusion's `DecorrelateLateralJoin` rule handles the general case (outer references in filters and aggregates), but not the simple case where an outer reference appears only in a `Projection`:

```
SELECT *, (SELECT t1.a + 1)
FROM t1
LATERAL VIEW explode(arr) tmp AS val
```

Sail's `DecorrelateLateralProjection` handles this simpler case first, rewriting it into a `CrossJoin + Projection` before DataFusion's more expensive rule runs. Sail's rule is *prepended* to DataFusion's list because it must run before `DecorrelateLateralJoin`:

```
// crates/sail-logical-optimizer/src/lib.rs
pub fn default_optimizer_rules() -> Vec<Arc<dyn OptimizerRule + Send + Sync>> {
    let Optimizer { rules } = Optimizer::default();
    // Custom rules are prepended so they run before DataFusion's built-in rules.
    // `DecorrelateLateralProjection` must run before `DecorrelateLateralJoin`
    // because it handles the simple case where OuterRef only appears in
    // Projection expressions (e.g. `LATERAL (SELECT t1.a + 1)`), rewriting
    // it into a CrossJoin + Projection. The remaining complex cases (OuterRef
    // in Filter/Aggregate) are left for DataFusion's `DecorrelateLateralJoin`.
    let mut custom: Vec<Arc<dyn OptimizerRule + Send + Sync>> =
        vec![Arc::new(DecorrelateLateralProjection::new())];
    custom.extend(rules);
    custom
}
```

`ExpandRowLevelOp` (Logical, in `sail-plan-lakehouse`)

There is a second logical optimizer rule for Delta/Iceberg write operations: `ExpandRowLevelOp` in `crates/sail-plan-lakehouse/src/optimizer.rs`. It rewrites `MergeIntoNode` and `FileDeleteNode` for lakehouse formats into `RowLevelWriteNode`, which routes to format-specific physical planners:

```
// crates/sail-plan-lakehouse/src/optimizer.rs
impl OptimizerRule for ExpandRowLevelOp {
    fn rewrite(&self, plan: LogicalPlan, _config: &dyn OptimizerConfig)
        -> Result<Transformed<LogicalPlan>>
    {
        plan.transform_up(|plan| {
            if let LogicalPlan::Extension(ext) = &plan {
                // MERGE expansion for lakehouse formats
                if let Some(node) = ext.node.as_any().downcast_ref::<MergeIntoNode>()
{
                    if !is_lakehouse_format(&node.options().target.format) {
                        return Ok(Transformed::no(plan));
```

```
                    }
                    return expand_merge_node(node);
                }
                // DELETE → RowLevelWriteNode for lakehouse formats
                if let Some(node) =
ext.node.as_any().downcast_ref::<FileDeleteNode>() {
                    if !is_lakehouse_format(node.options().format.as_str()) {
                        return Ok(Transformed::no(plan));
                    }
                    return expand_delete_node(node);
                }
            }
            Ok(Transformed::no(plan))
        })
    }
}
```

Non-lakehouse DELETE and MERGE fall through to the standard file-based planners.

The Physical Optimizer

The physical optimizer (`sail-physical-optimizer`) does not prepend to DataFusion's default pipeline. It reconstructs the **entire** physical optimizer pipeline from scratch, embedding all DataFusion rules at their canonical positions and inserting Sail's custom rules after them:

```
// crates/sail-physical-optimizer/src/lib.rs
pub fn get_physical_optimizers(options: PhysicalOptimizerOptions)
    -> Vec<Arc<dyn PhysicalOptimizerRule + Send + Sync>>
{
    let mut rules: Vec<Arc<dyn PhysicalOptimizerRule + Send + Sync>> = vec![];

    rules.push(Arc::new(OutputRequirements::new_add_mode()));
    rules.push(Arc::new(AggregateStatistics::new()));
    if options.enable_join_reorder {
        rules.push(Arc::new(JoinReorder::new(options.join_reorder)));
    }
    rules.push(Arc::new(JoinSelection::new()));
    rules.push(Arc::new(LimitedDistinctAggregation::new()));
    rules.push(Arc::new(FilterPushdown::new()));
    rules.push(Arc::new(EnforceDistribution::new()));
    rules.push(Arc::new(CombinePartialFinalAggregate::new()));
    rules.push(Arc::new(EnforceSorting::new()));
    rules.push(Arc::new(OptimizeAggregateOrder::new()));
    rules.push(Arc::new(ProjectionPushdown::new()));
    rules.push(Arc::new(OutputRequirements::new_remove_mode()));
    rules.push(Arc::new(TopKAggregation::new()));
    rules.push(Arc::new(LimitPushPastWindows::new()));
    rules.push(Arc::new(LimitPushdown::new()));
    rules.push(Arc::new(ProjectionPushdown::new()));
    rules.push(Arc::new(PushdownSort::new()));
    rules.push(Arc::new(EnsureCooperative::new()));
    rules.push(Arc::new(FilterPushdown::new_post_optimization()));
    // --- Sail custom rules, run after all DataFusion rules ---
    rules.push(Arc::new(RewriteExplicitRepartition::new()));
    rules.push(Arc::new(RewriteCollectLeftHashJoin::new()));
    rules.push(Arc::new(EnforceBarrierPartitioning::new()));
    rules.push(Arc::new(SanityCheckPlan::new()));
```

```
    rules
}
```

A test verifies that the DataFusion rule names appear in exactly the same order as in DataFusion's own `PhysicalOptimizer::default()`, so any DataFusion rule additions or reorderings are caught at compile time.

The four custom physical optimizer rules:

Rule	Purpose
`JoinReorder`	DP-based join reorder: cardinality estimation, cost model, `n ≤ max_relations` constraint. The "join reorder safeguards" from commit #1954 are options on this rule.
`RewriteExplicitRepartition`	Converts `ExplicitRepartitionExec` (placeholder) into the correct DataFusion exec: `RepartitionExec` (hash), `CoalescePartitionsExec` (coalesce-to-1), or passthrough.
`RewriteCollectLeftHashJoin`	Safety net: ensures the build side of every `HashJoinExec` in `CollectLeft` mode has exactly one output partition (inserts `CoalescePartitionsExec` if violated).
`EnforceBarrierPartitioning`	Ensures `BarrierExec` nodes have the correct partitioning for streaming checkpoints.

The Custom Node Inventory and `sail-session`

There are 17 custom `UserDefinedLogicalNodeCore` implementations across `sail-logical-plan`. Every one needs a corresponding physical plan, and that dispatch lives in a single place: `ExtensionPhysicalPlanner` in `crates/sail-session/src/planner.rs`.

`sail-session`: `ExtensionQueryPlanner` and `ExtensionPhysicalPlanner`

`sail-session` is the crate that assembles the complete physical planning pipeline and registers it with DataFusion's `SessionStateBuilder`. The key struct is `ExtensionQueryPlanner`:

```
// crates/sail-session/src/planner.rs
#[async_trait]
impl QueryPlanner for ExtensionQueryPlanner {
    async fn create_physical_plan(
        &self,
        logical_plan: &LogicalPlan,
        session_state: &SessionState,
    ) -> datafusion::common::Result<Arc<dyn ExecutionPlan>> {
        let mut extension_planners = new_lakehouse_extension_planners();
        extension_planners.push(Arc::new(SystemTablePhysicalPlanner));
        extension_planners.push(Arc::new(ListingTablePhysicalPlanner));
        extension_planners.push(Arc::new(ExtensionPhysicalPlanner));
        let planner =
DefaultPhysicalPlanner::with_extension_planners(extension_planners);
        planner.create_physical_plan(&logical_plan, session_state).await
    }
}
```

Four extension planners are chained in order:

Planner	Handles
`DeltaTablePhysicalPlanner`	Delta Lake table scans
`IcebergTablePhysicalPlanner`	Iceberg table scans
`DeltaExtensionPlanner`	Delta write/delete/merge logical nodes
`SystemTablePhysicalPlanner`	System catalog table sources
`ListingTablePhysicalPlanner`	Parquet/CSV/JSON/ORC/Avro file listings
`ExtensionPhysicalPlanner`	All 17 `UserDefinedLogicalNodeCore` nodes

`ExtensionPhysicalPlanner::plan_extension` is a chain of `downcast_ref` checks — one per custom node type:

```
// crates/sail-session/src/planner.rs
impl ExtensionPlanner for ExtensionPhysicalPlanner {
    async fn plan_extension(&self, planner, node, logical_inputs, physical_inputs,
session_state)
        -> Result<Option<Arc<dyn ExecutionPlan>>>
    {
        let plan = if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
            Arc::new(RangeExec::try_new(node.range().clone(),
node.num_partitions(), ...)?)
        } else if let Some(node) = node.as_any().downcast_ref::<ShowStringNode>() {
            Arc::new(ShowStringExec::new(input, node.names().to_vec(),
node.limit(), ...))
        } else if let Some(node) = node.as_any().downcast_ref::<MapPartitionsNode>()
{
            Arc::new(MapPartitionsExec::new(input, node.udf().clone(), ...))
        } else if let Some(node) = node.as_any().downcast_ref::<MonotonicIdNode>() {
            Arc::new(MonotonicIdExec::try_new(input,
node.column_name().to_string(), ...)?)
        } else if let Some(node) =
node.as_any().downcast_ref::<SparkPartitionIdNode>() {
            Arc::new(SparkPartitionIdExec::try_new(input, node.column_name(), ...)?)
        } else if let Some(node) =
node.as_any().downcast_ref::<SortWithinPartitionsNode>() {
            let sort = SortExec::new(ordering,
input).with_preserve_partitioning(true);
            Arc::new(sort)
        } else if let Some(node) = node.as_any().downcast_ref::<SchemaPivotNode>() {
            Arc::new(SchemaPivotExec::new(input, node.names().to_vec(), ...))
        } else if let Some(node) = node.as_any().downcast_ref::<FileWriteNode>() {
            create_file_write_physical_plan(session_state, planner,
logical_input, ...).await?
        } else if let Some(node) = node.as_any().downcast_ref::<FileDeleteNode>() {
            create_file_delete_physical_plan(session_state, planner,
schema, ...).await?
        } else if let Some(_node) = node.as_any().downcast_ref::<MergeIntoNode>() {
            return internal_err!("MERGE expects pre-expanded plan
(RowLevelWriteNode)")
        } else if let Some(node) =
node.as_any().downcast_ref::<ExplicitRepartitionNode>() {
            Arc::new(ExplicitRepartitionExec::new(input, partitioning))
        } else if node.as_any().is::<StreamSourceAdapterNode>() {
```

```
            Arc::new(StreamSourceAdapterExec::new(input))
        } else if let Some(node) =
node.as_any().downcast_ref::<StreamSourceWrapperNode>() {
            node.source().scan(session_state, ...).await?
        } else if let Some(node) = node.as_any().downcast_ref::<StreamLimitNode>() {
            Arc::new(StreamLimitExec::try_new(input, node.skip(), node.fetch())?)
        } else if let Some(node) = node.as_any().downcast_ref::<StreamFilterNode>() {
            Arc::new(StreamFilterExec::try_new(input, predicate)?)
        } else if node.as_any().is::<StreamCollectorNode>() {
            Arc::new(StreamCollectorExec::try_new(input)?)
        } else if let Some(node) = node.as_any().downcast_ref::<CatalogCommandNode>()
{
            Arc::new(CatalogCommandExec::new(node.command().clone(), schema))
        } else if let Some(_node) = node.as_any().downcast_ref::<BarrierNode>() {
            Arc::new(BarrierExec::new(preconditions.to_vec(), plan.clone()))
        } else {
            return internal_err!("unsupported logical extension node: {:?}", node);
        };
        Ok(Some(plan))
    }
}
```

The complete set of 17 custom logical nodes and their physical counterparts:

Logical node	Physical plan	Purpose
`RangeNode`	`RangeExec`	`spark.range(start, end, step, n)`
`ShowStringNode`	`ShowStringExec`	`df.show()` — collects to one partition, formats table string
`MapPartitionsNode`	`MapPartitionsExec`	Python/Scala UDFs via `mapInPandas`, `mapInArrow`
`MonotonicIdNode`	`MonotonicIdExec`	`monotonically_increasing_id()`
`SparkPartitionIdNode`	`SparkPartitionIdExec`	`spark_partition_id()`
`SortWithinPartitionsNode`	`SortExec(preserve_partitioning=true)`	`sortWithinPartitions()`
`SchemaPivotNode`	`SchemaPivotExec`	Schema pivoting for UNPIVOT
`FileWriteNode`	`create_file_write_physical_plan()`	All file writes (Parquet, CSV, Delta, ...)
`FileDeleteNode`	`create_file_delete_physical_plan()`	DELETE statement
`MergeIntoNode`	*(pre-expanded by `ExpandRowLevelOp`)*	MERGE INTO (must be rewritten first)
`ExplicitRepartitionNode`	`ExplicitRepartitionExec`	`repartition()`, `coalesce()`, `repartitionByRange()`
`StreamSourceAdapterNode`	`StreamSourceAdapterExec`	Adapts batch source for streaming plan
`StreamSourceWrapperNode`	Direct scan from `StreamSource`	Wraps a streaming data source
`StreamLimitNode`	`StreamLimitExec`	`LIMIT`/`OFFSET` on streaming plan
`StreamFilterNode`	`StreamFilterExec`	`WHERE` predicate on streaming plan
`StreamCollectorNode`	`StreamCollectorExec`	Collects streaming output into sink
`CatalogCommandNode`	`CatalogCommandExec`	DDL operations (CREATE TABLE, etc.)
`BarrierNode`	`BarrierExec`	Streaming checkpoint barrier

`PlanProperties`: The Physical Contract

Every `ExecutionPlan` implementation must declare its `PlanProperties` — the metadata about its output:

```
pub struct PlanProperties {
    eq_properties: EquivalenceProperties,  // sort order, uniqueness
    partitioning:  Partitioning,           // output partition scheme
    emission_type: EmissionType,           // Final or Streaming
    boundedness:   Boundedness,            // Bounded or Unbounded
}
```

For example, `ShowStringExec` collects all data into a single partition and emits only when the stream is fully consumed:

```
let properties = Arc::new(PlanProperties::new(
    EquivalenceProperties::new(schema.clone()),
```

```
    Partitioning::RoundRobinBatch(1),  // single output partition
    EmissionType::Final,               // output only after all input consumed
    Boundedness::Bounded,
));
```

The `required_input_distribution` method returns `Distribution::SinglePartition`, which signals DataFusion to inject a `CoalescePartitionsExec` before `ShowStringExec` when the input is multi-partitioned.

The Session Extension Mechanism

Sail needs to store per-session state (Spark configuration, job runner, streaming queries) alongside DataFusion's `SessionContext`. DataFusion's extension map provides this:

```
// DataFusion's SessionConfig
pub fn with_extension<T: Any + Send + Sync + 'static>(
    mut self,
    ext: Arc<T>,
) -> Self { /* inserts into HashMap<TypeId, Arc<dyn Any>> */ }
```

Sail wraps this with a typed accessor trait `SessionExtensionAccessor`:

```
pub trait SessionExtensionAccessor {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
}

impl SessionExtensionAccessor for SessionContext {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>> {
        self.state()
            .config()
            .get_extension::<T>()
            .ok_or_else(|| /* error */ )
    }
}
```

This is used throughout `sail-spark-connect` and `sail-flight` to retrieve the `SparkSession` or `JobService` from a `&SessionContext`:

```
let spark = ctx.extension::<SparkSession>()?;
let plan_config = spark.plan_config()?;
```

DataFusion Catalogs

DataFusion has a catalog/schema/table hierarchy. Sail maps Spark's catalog model onto DataFusion's by implementing `CatalogProvider`, `SchemaProvider`, and `TableProvider` traits for each catalog backend. The catalog abstraction layer is in `crates/sail-catalog/`; specific implementations are in `sail-catalog-memory`, `sail-catalog-glue`, etc. This is covered in detail in Chapter 7.

Summary

Sail uses DataFusion as its planning and execution backbone and extends it at six distinct layers:

1. **Custom logical nodes** (17 `UserDefinedLogicalNodeCore` impls in `sail-logical-plan`): `RangeNode`, `MapPartitionsNode`, `FileWriteNode`, `BarrierNode`, 5 streaming nodes, and more.
2. **Custom physical nodes** (`ExecutionPlan` in `sail-physical-plan`): one per custom logical node — 17 physical counterparts.
3. **Logical optimizer rules** (`sail-logical-optimizer`, `sail-plan-lakehouse`): `DecorrelateLateralProjection` prepended before DataFusion's rules; `ExpandRowLevelOp` for lakehouse write expansion.

4. **Physical optimizer** (`sail-physical-optimizer`): rebuilds the entire pipeline from scratch, inserting custom rules (`JoinReorder`, `RewriteExplicitRepartition`, `RewriteCollectLeftHashJoin`, `EnforceBarrierPartitioning`) after all DataFusion rules.
5. **Extension planners** (`sail-session`): `ExtensionQueryPlanner` chains 6 `ExtensionPlanner` implementations; `ExtensionPhysicalPlanner` dispatches all 17 custom nodes.
6. **Session extensions**: `SparkSession`, `PlanService`, `JobService` attached to DataFusion's `SessionContext` type-map.

The `PlanResolver` is the translation layer between Sail's `spec::Plan` IR and DataFusion's `LogicalPlan`. Everything below `PlanResolver` is DataFusion-native code; everything above it is Spark-specific. The `sail-session` crate is the assembly point where all custom extensions are wired into the DataFusion session state.

Chapter 5: Apache Arrow Flight SQL

Two Entry Points

Sail exposes two gRPC services on two different ports (by default):

- **Spark Connect** (`:50051`) — the primary interface, designed for PySpark clients
- **Arrow Flight SQL** (`:50052`) — a secondary interface for any ADBC-compatible client, JDBC driver, or tool that speaks Arrow Flight natively

Spark Connect is described in Chapter 2. This chapter covers Flight SQL.

What Is Arrow Flight?

Arrow Flight is a gRPC-based protocol designed specifically for bulk data transfer of Arrow data. Regular gRPC uses Protobuf for both control messages and data; Flight uses Protobuf for control messages but Arrow IPC for the data payload. This means there is no per-row serialization overhead for result data — the columnar bytes flow from server to client with minimal transformation.

Arrow Flight defines two foundational operations: - **`DoGet(Ticket) → stream<FlightData>`**: given a ticket, stream Arrow data back - **`DoPut(stream<FlightData>) → stream<PutResult>`**: stream Arrow data to the server

Arrow Flight SQL layers a SQL query interface on top of Flight. The key addition is a two-phase protocol:

1. **`GetFlightInfo(CommandStatementQuery)`** ⟶ `FlightInfo`: parse and plan the query, return a `FlightInfo` containing a `Ticket` (an opaque handle to the planned query)
2. **`DoGet(Ticket)`** ⟶ stream of `FlightData`: execute the planned query and stream results

This two-phase design allows clients to inspect the schema *before* fetching data (the `FlightInfo` includes the output schema), and it allows the server to pipeline execution: planning and execution can overlap with data transfer.

`SailFlightSqlService`

Sail implements Arrow Flight SQL in `crates/sail-flight/src/service.rs`:

```
pub struct SailFlightSqlService {
    session_manager: SessionManager,
    config: Arc<PlanConfig>,
    metrics: Option<Arc<MetricRegistry>>,
    state: Arc<Mutex<SailFlightSqlState>>,
}
```

```
#[tonic::async_trait]
impl FlightSqlService for SailFlightSqlService {
    type FlightService = SailFlightSqlService;
    // ...
}
```

`FlightSqlService` is the trait from the `arrow-flight` crate. Sail only needs to implement the methods it supports; the rest default to `UNIMPLEMENTED`.

Phase 1: Planning (`get_flight_info_statement`)

When a client sends `SELECT avg(amount) FROM orders`, it arrives as a `CommandStatementQuery` in `get_flight_info_statement`:

```
async fn get_flight_info_statement(
    &self,
    query: CommandStatementQuery,
    request: Request<FlightDescriptor>,
) -> Result<Response<FlightInfo>, Status> {
    // Parse SQL text to AST
    let statement = parse_one_statement(&query.query)
        .map_err(|e| Status::invalid_argument(format!("parse error: {e}")))?;

    // Convert AST to spec::Plan IR
    let plan = from_ast_statement(statement)
        .map_err(|e| Status::invalid_argument(format!("plan conversion error:
{e}")))?;

    // Get (or create) the session context
    let ctx = self.get_session_context().await?;

    // Resolve, optimize, and create a physical plan; get back a stream handle
    let (plan, _) = resolve_and_execute_plan(&ctx, self.config.clone(), plan)
        .await
        .map_err(|e| Status::internal(format!("plan error: {e}")))?;

    let schema = plan.schema();
    let service = ctx.extension::<JobService>()?;
    let stream = service.runner().execute(&ctx, plan).await?;

    // Wrap stream in metrics if enabled
    let stream = if let Some(ref m) = self.metrics {
        Box::pin(MetricsRecordingStream::new(stream, m.clone(), ctx))
    } else {
        stream
    };

    // Store the stream under an opaque handle
    let handle = QueryHandle::new();
    self.state.lock().await.add_stream(handle.clone(), stream);

    // Package the handle as a Flight Ticket
    let ticket = TicketStatementQuery {
        statement_handle: handle.as_bytes().to_vec().into(),
    };
    let ticket_bytes = ticket.as_any().encode_to_vec();

    // Return FlightInfo with the schema and the ticket
```

```
    let endpoint = FlightEndpoint {
        ticket: Some(Ticket { ticket: ticket_bytes.into() }),
        location: vec![],
        expiration_time: None,
        app_metadata: Default::default(),
    };
    let info = FlightInfo::new()
        .with_endpoint(endpoint)
        .with_descriptor(request.into_inner())
        .try_with_schema(&schema)?;

    Ok(Response::new(info))
}
```

The notable design decision: Sail starts execution *eagerly* in `get_flight_info_statement`. The `service.runner().execute(...)` call creates the `SendableRecordBatchStream` and spawns the execution. The stream is stored in `SailFlightSqlState` (a `HashMap<QueryHandle, SendableRecordBatchStream>` behind a `Mutex<...>`), keyed by the handle. The client gets back the schema and a ticket immediately; the query is already running.

This is different from the Spark Connect path, where execution starts only when the client calls `ExecutePlan` and begins consuming the streaming response. Flight SQL's two-phase design means execution must start at phase 1, because the `FlightInfo` cannot include the output schema without having resolved the plan.

Phase 2: Streaming (`do_get_statement`)

When the client presents the ticket, it calls `DoGet`:

```
async fn do_get_statement(
    &self,
    ticket: TicketStatementQuery,
    _request: Request<Ticket>,
) -> Result<Response<<Self as FlightService>::DoGetStream>, Status> {
    let handle = QueryHandle::try_from(ticket.statement_handle.as_ref())?;

    // Remove the stream from the state map (consume once)
    let stream = self
        .state
        .lock()
        .await
        .remove_stream(&handle)
        .ok_or_else(|| Status::not_found(
            format!("query handle not found or already consumed: {handle}")
        ))?;

    let schema = stream.schema();

    // Convert RecordBatch errors to FlightError
    let output = stream.map(|result| {
        result.map_err(|e|
arrow_flight::error::FlightError::ExternalError(Box::new(e)))
    });

    // Encode as Arrow IPC Flight frames
    let output = FlightDataEncoderBuilder::new()
        .with_schema(schema)
```

```
        .build(output)
        .map(|result| result.map_err(|e| Status::internal(format!("encoding error:
{e}"))));

    Ok(Response::new(Box::pin(output)))
}
```

`FlightDataEncoderBuilder` from `arrow-flight` handles the encoding: it takes a `Stream<Item = Result<RecordBatch, ArrowError>>` and produces a `Stream<Item = Result<FlightData, FlightError>>`. Each `FlightData` message is an Arrow IPC frame that the client can decode directly with `pyarrow` or any ADBC driver.

The `remove_stream` semantics — the handle is consumed once — mean that each query can only be fetched once. If the client's network fails after the ticket is issued but before `DoGet` is called, the query is lost. This is acceptable for the Flight SQL use case (re-execute on failure) and avoids indefinite server-side memory growth.

The Session Model for Flight SQL

Unlike Spark Connect, which creates one `SessionContext` per client session, the Flight SQL service uses a single shared session:

```
impl SailFlightSqlService {
    const DEFAULT_SESSION_ID: &'static str = "flight-default";
    const DEFAULT_USER_ID: &'static str = "flight-user";

    async fn get_session_context(&self) -> Result<SessionContext, Status> {
        self.session_manager
            .get_or_create_session_context(
                Self::DEFAULT_SESSION_ID.to_string(),
                Self::DEFAULT_USER_ID.to_string(),
            )
            .await
            .map_err(|e| Status::internal(format!("session error: {e}")))
    }
}
```

This is a deliberate simplification for the initial Flight SQL implementation. All Flight SQL queries share one session, which means they share the same configuration and catalog state. A future version could multiplex sessions using the `CallHeaders` mechanism from Flight (which can carry authentication tokens or session IDs).

Handshake

Flight's `DoHandshake` is used for authentication. Sail's implementation is minimal:

```
async fn do_handshake(
    &self,
    _request: Request<Streaming<HandshakeRequest>>,
) -> Result<Response<Pin<Box<dyn Stream<Item = Result<HandshakeResponse, Status>> +
Send>>>, Status>
{
    debug!("handshake received from client");
    let response = HandshakeResponse {
        protocol_version: 0,
        payload: Default::default(),
    };
    let output = stream::iter(vec![Ok(response)]);
```

```
    Ok(Response::new(Box::pin(output)))
}
```

The `// Note: not all clients perform handshake with the server.` comment in the source is informative: some Flight SQL clients (e.g. ADBC with certain drivers) skip the handshake and go directly to `GetFlightInfo`. Sail accepts both patterns.

Metrics and Observability

When OpenTelemetry is enabled at server startup, the Flight SQL service wraps result streams with `MetricsRecordingStream`:

```
let stream: SendableRecordBatchStream = if let Some(ref m) = self.metrics {
    Box::pin(MetricsRecordingStream::new(
        stream,
        m.clone(),
        MetricsRecordingContext { statement_type },
    ))
} else {
    stream
};
```

`MetricsRecordingStream` is a transparent wrapper that tracks row counts, batch counts, and elapsed time, recording them to the OpenTelemetry `MetricRegistry` when the stream is fully consumed or dropped. The `StatementType` discriminates between `Query` (SELECT) and `Command` (DDL, DML) for metric labels.

Command Execution

Flight SQL commands (DDL, INSERT, etc.) need special handling because they produce no rows. Sail handles this by eagerly draining the stream and returning an empty result:

```
let stream: SendableRecordBatchStream = match statement_type {
    StatementType::Query => stream,
    StatementType::Command => {
        // Execute command eagerly and store the result in memory
        let mut stream = stream;
        let mut batches = Vec::new();
        while let Some(result) = stream.next().await {
            batches.push(result.map_err(|e| Status::internal(...))?);
        }
        Box::pin(RecordBatchStreamAdapter::new(
            schema.clone(),
            stream::iter(batches.into_iter().map(Ok)),
        ))
    }
};
```

This ensures that DDL commands run to completion before `get_flight_info_statement` returns, so the client can trust that the command succeeded when it receives the `FlightInfo`. The result stream in state will be empty (zero batches) but still present, which the client can fetch or ignore.

Comparison: Flight SQL vs Spark Connect

Aspect	Spark Connect	Flight SQL
Protocol	Spark-specific protobuf	Arrow Flight RPC standard
Session model	Per-session state, configurable	Single shared session
Streaming	Native reattachable streaming	Two-phase (info + fetch)
Execution start	On first stream poll	During `GetFlightInfo`
Error reattach	`ReattachExecute` RPC	Re-execute
Primary use	PySpark / pyspark-client	ADBC, JDBC, BI tools

Summary

`sail-flight` provides an Arrow Flight SQL entry point alongside Spark Connect. Its implementation follows the standard two-phase pattern: plan eagerly in `get_flight_info_statement`, stream results in `do_get_statement`. Execution starts at phase 1; the resulting stream is stored in a handle map and consumed once when the client fetches it. This makes Sail accessible to any Flight SQL-compatible client — DuckDB, Tableau, Apache Superset — without requiring PySpark.

Chapter 6: The Execution Layer

The JobRunner Abstraction

After `resolve_and_execute_plan` produces an `Arc<dyn ExecutionPlan>`, that tree must actually run. Sail abstracts execution behind a `JobRunner` trait in `sail-common-datafusion`:

```
#[tonic::async_trait]
pub trait JobRunner: Send + Sync {
    async fn execute(
        &self,
        ctx: &SessionContext,
        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream>;

    async fn stop(&self, history: oneshot::Sender<JobRunnerHistory>);
}
```

There are two implementations:

Implementation	When used
`LocalJobRunner`	Single-process mode (development, testing, small data)
`ClusterJobRunner`	Distributed mode (driver/worker cluster)

The session chooses which backend to use based on configuration. Both implement the same trait, so all planning and protocol code is identical — the execution backend is swapped transparently.

`LocalJobRunner`: Execution in One Process

`LocalJobRunner` is the simplest path. It just calls DataFusion's `execute_stream`:

```
// crates/sail-execution/src/job_runner.rs
#[tonic::async_trait]
impl JobRunner for LocalJobRunner {
    async fn execute(
```

```
        &self,
        ctx: &SessionContext,
        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream> {
        if self.stopped.load(Ordering::Relaxed) {
            return internal_err!("job runner is stopped");
        }
        let job_id = self.next_job_id.fetch_add(1, Ordering::Relaxed);
        let options = TracingExecOptions {
            metrics: global_metrics(),
            job_id: Some(job_id),
            stage: None,
            attempt: None,
            operator_id: None,
        };
        let plan = trace_execution_plan(plan, options)?;
        Ok(execute_stream(plan, ctx.task_ctx())?)
    }
}
```

`trace_execution_plan` wraps the plan tree with OpenTelemetry tracing spans so execution metrics flow to the configured exporter. Then `execute_stream` (from `datafusion`) produces a `SendableRecordBatchStream` that drives execution lazily as the consumer polls the stream.

In local mode, all partitions of all physical plan operators run on the Tokio thread pool of the current process. DataFusion's `execute_stream` handles partition fan-out and merge internally.

`ClusterJobRunner`: Distributed Execution

`ClusterJobRunner` is the path for multi-node execution. It wraps a `DriverActor` handle:

```
pub struct ClusterJobRunner {
    driver: ActorHandle<DriverActor>,
}

#[tonic::async_trait]
impl JobRunner for ClusterJobRunner {
    async fn execute(
        &self,
        ctx: &SessionContext,
        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream> {
        let (tx, rx) = oneshot::channel();
        self.driver
            .send(DriverEvent::ExecuteJob {
                plan,
                context: ctx.task_ctx(),
                result: tx,
            })
            .await
            .map_err(|e| internal_datafusion_err!("{e}"))?;
        rx.await
            .map_err(|e| internal_datafusion_err!("failed to create job stream:
{e}"))?
            .map_err(|e| internal_datafusion_err!("{e}"))
    }
}
```

`execute` sends a `DriverEvent::ExecuteJob` message to the driver actor and awaits the `oneshot::Receiver` for the resulting stream. The driver does not block the caller — it processes the event asynchronously and sends the stream back through the oneshot channel. From the caller's perspective, `execute` returns as soon as the driver has created the stream; actual execution happens as the stream is polled.

The Actor Model

Sail's execution layer is built on an actor model implemented in `crates/sail-server/src/actor.rs`. Actors are Tokio tasks that own mutable state and receive messages sequentially through an mpsc channel. No mutexes; no shared mutable state between actors.

The `Actor` Trait

```
#[tonic::async_trait]
pub trait Actor: Sized + Send + 'static {
    type Message: Send + SpanAssociation + 'static;
    type Options;

    fn name() -> &'static str;
    fn new(options: Self::Options) -> Self;
    async fn start(&mut self, ctx: &mut ActorContext<Self>) {}
    fn receive(&mut self, ctx: &mut ActorContext<Self>, message: Self::Message) ->
ActorAction;
    async fn stop(self, ctx: &mut ActorContext<Self>) {}
}
```

The `receive` method is *synchronous* — it must not block. If the actor needs to perform async work (e.g. send an RPC to a worker), it uses `ctx.spawn(...)` to launch a separate task that sends the result back as another message.

The `ActorRunner` drives the actor:

```
impl<T: Actor> ActorRunner<T> {
    async fn run(mut self) {
        self.actor.start(&mut self.ctx).await;
        while let Some(MessageEnvelop { message, context }) =
self.receiver.recv().await {
            let action = self.actor.receive(&mut self.ctx, message);
            match action {
                ActorAction::Continue => {}
                ActorAction::Stop => break,
            }
            self.ctx.reap();  // join completed child tasks, log errors
        }
        self.receiver.close();
        self.actor.stop(&mut self.ctx).await;
    }
}
```

The event loop processes messages one at a time. All actor state is `&mut self` in `receive`, so there are no data races. Tracing spans are carried through the `MessageEnvelop` struct:

```
struct MessageEnvelop<M> {
    message: M,
    context: Option<SpanContext>,
}
```

When an actor sends a message via `ActorHandle::send`, it creates a span with `Span::enter_with_local_parent` and attaches the `SpanContext` to the envelope. When the receiving actor processes it, it creates a child span, connecting the trace across actor boundaries.

`DriverActor`

The `DriverActor` is the central coordinator for distributed execution. Its state includes:

```
pub struct DriverActor {
    options: DriverOptions,
    server: ServerMonitor,         // the driver's gRPC server for workers to connect
to
    worker_pool: WorkerPool,       // registered workers and their health state
    job_scheduler: JobScheduler,   // pending and active jobs
    task_assigner: TaskAssigner,   // assigns tasks to workers
    task_runner: TaskRunner,       // executes driver-side tasks directly
    stream_manager: StreamManager, // manages inter-stage data streams
    task_sequences: HashMap<TaskKey, u64>,
    history: Option<oneshot::Sender<JobRunnerHistory>>,
}
```

The driver handles the events defined in `DriverEvent`:

```
pub enum DriverEvent {
    ServerReady { port: u16, signal: oneshot::Sender<()> },
    RegisterWorker { worker_id: WorkerId, host: String, port: u16, result:
oneshot::Sender<ExecutionResult<()>> },
    WorkerHeartbeat { worker_id: WorkerId },
    ExecuteJob { plan: Arc<dyn ExecutionPlan>, context: Arc<TaskContext>, result:
oneshot::Sender<ExecutionResult<SendableRecordBatchStream>> },
    UpdateTask { key: TaskKey, status: TaskStatus, message: Option<String>, cause:
Option<CommonErrorCause>, sequence: Option<u64> },
    CreateLocalStream { key: TaskStreamKey, storage: LocalStreamStorage, schema:
SchemaRef, result: oneshot::Sender<ExecutionResult<Box<dyn TaskStreamSink>>> },
    FetchWorkerStream { worker_id: WorkerId, key: TaskStreamKey, schema: SchemaRef,
result: oneshot::Sender<ExecutionResult<TaskStreamSource>> },
    Shutdown { history: Option<oneshot::Sender<JobRunnerHistory>> },
    // ... more
}
```

The Job Graph

When `ExecuteJob` arrives, the driver builds a `JobGraph` by analyzing the `ExecutionPlan` tree. The job graph partitions the plan into *stages*, where stage boundaries are exchange operators (hash-repartition, broadcast, sort-merge join shuffles):

```
/// A job graph represents a distributed execution plan for a job.
/// A job consists of multiple *stages*, where each stage has one or more
/// *partitions*. There are *tasks* which each corresponds to the execution of a
single partition
/// of a stage and can have multiple *attempts*.
/// Each task produces output split into multiple *channels*.
pub struct JobGraph {
    stages: Vec<Stage>,  // topologically sorted
    schema: SchemaRef,
}

pub struct Stage {
    pub inputs: Vec<StageInput>,
```

```
    pub plan: Arc<dyn ExecutionPlan>,
    pub group: String,    // slot sharing group for co-location
    pub mode: OutputMode,
    pub distribution: OutputDistribution,
    pub placement: TaskPlacement,  // Driver or Worker
}
```

Stages are topologically sorted: all inputs of a stage appear before it in the list. `TaskPlacement::Driver` marks stages that must run on the driver node — typically catalog operations, `SHOW` commands, and other non-data-parallel operators.

Input Modes

The relationship between stages is described by `InputMode`:

```
pub enum InputMode {
    /// partition p of the current stage reads from partition p of the input stage
    Forward,
    /// each partition of the current stage reads from all partitions of the input
stage
    Merge,
    /// partition p reads from channel p of all partitions in the input stage (hash
shuffle read)
    Shuffle,
    /// a single partition reads from all partitions of the input stage (broadcast)
    Broadcast,
    /// each partition reads from a contiguous subset of input partitions (coalesce)
    Rescale,
}
```

And output distribution:

```
pub enum OutputDistribution {
    Hash { keys: Vec<Arc<dyn PhysicalExpr>>, channels: usize },
    RoundRobin { channels: usize },
    RoundRobinRow { channels: usize },  // row-level, for explicit df.repartition()
}
```

The planner assigns `InputMode::Shuffle` to the output of a hash-repartition stage and `InputMode::Forward` to the output of a map (projection, filter) stage. This determines how inter-stage data is routed.

Task Identification

Tasks are identified by a composite key:

```
pub struct TaskKey {
    pub job_id: JobId,
    pub stage: usize,
    pub partition: usize,
    pub attempt: usize,
}
```

Inter-stage data streams add a channel dimension:

```
pub struct TaskStreamKey {
    pub job_id: JobId,
    pub stage: usize,
    pub partition: usize,
    pub attempt: usize,
```

```
    pub channel: usize,  // for hash-distributed outputs
}
```

`TaskDefinition` is the serializable description of what a task should execute — the physical plan encoded as bytes (via `datafusion_proto`), plus input and output routing:

```
pub struct TaskDefinition {
    pub plan: Arc<[u8]>,       // protobuf-encoded ExecutionPlan subtree
    pub inputs: Vec<TaskInput>,
    pub output: TaskOutput,
}

pub struct TaskOutput {
    pub distribution: TaskOutputDistribution,  // Hash, RoundRobin, or RoundRobinRow
    pub locator: TaskOutputLocator,            // Local or Remote { uri }
}
```

Tasks are serialized and sent to workers over gRPC. The worker deserializes the `plan` bytes back into an `ExecutionPlan` using DataFusion's protobuf codec.

Task Scheduling and Regions

The scheduler groups tasks into `TaskRegions` — collections of tasks that must be scheduled and rescheduled together:

```
pub struct TaskRegion {
    pub tasks: Vec<(TaskPlacement, TaskSet)>,
}

pub struct TaskSet {
    pub entries: Vec<TaskSetEntry>,
}

pub struct TaskSetEntry {
    pub key: TaskKey,
    pub output: TaskOutputKind,  // Local or Remote
}
```

A region typically corresponds to a pipeline of pipelined stages: stages whose outputs are pipelined (not blocking) can run concurrently on the same worker because they produce and consume data incrementally. This reduces intermediate materialization.

When a task fails, the entire region is rescheduled, because a failure in one pipelined stage may have caused incomplete output in co-running stages.

Data Exchange Between Stages

Inter-stage data exchange uses three transport modes:

1. **Local streams**: data stays in the same process (driver mode or same-worker optimization). Stored in a `LocalStreamStorage` — either in memory (`Memory`) or on disk (`Disk`).
2. **Worker streams**: data is fetched from another worker via gRPC. The driver mediates the connection: a downstream worker asks the driver for the address of the upstream worker, then fetches directly.
3. **Remote streams**: data is stored at a URI (S3, GCS, Azure Blob) and fetched by URL. Used for data that must survive worker failure.

The stream manager tracks which streams exist and delivers them to tasks that request them.

Streaming Execution

Structured streaming is a separate execution path that runs alongside batch query execution. The entry point is the same `JobRunner`, but the logical plan is transformed before physical planning.

The Streaming Plan Rewriter

When `resolve_and_execute_plan` detects a streaming plan (`is_streaming_plan(&plan)?` returns true), it calls `rewrite_streaming_plan` before physical planning. This rewriter, in `crates/sail-plan/src/streaming/rewriter.rs`, converts the batch logical plan into a "flow event" plan:

```
// crates/sail-plan/src/streaming/rewriter.rs
impl StreamingRewriter {
    fn f_up_extension(&mut self, extension: Extension) ->
Result<Transformed<LogicalPlan>> {
        let node = extension.node.as_ref();
        if node.as_any().is::<RangeNode>() {
            // Wrap range source in a streaming adapter
            Ok(Transformed::yes(LogicalPlan::Extension(Extension {
                node: Arc::new(StreamSourceAdapterNode::try_new(Arc::new(
                    LogicalPlan::Extension(extension),
                ))?),
            })))
        } else if let Some(show) = node.as_any().downcast_ref::<ShowStringNode>() {
            // Wrap show_string's input in a collector (gathers all batches before
displaying)
            let input = LogicalPlan::Extension(Extension {
                node:
Arc::new(StreamCollectorNode::try_new(Arc::clone(show.input()))?),
            });
            Ok(Transformed::yes(LogicalPlan::Extension(Extension {
                node: show.with_exprs_and_inputs(vec![], vec![input])?,
            })))
        } else if node.as_any().is::<FileWriteNode>() {
            Ok(Transformed::no(LogicalPlan::Extension(extension)))  // write nodes
pass through
        } else {
            plan_err!("unsupported extension node for streaming: {node:?}")
        }
    }
}
```

Table scans are rewritten to use `StreamSourceWrapperNode`, which wraps a `StreamSource` trait object. The `StreamSource` trait is implemented by each streaming source (Kafka, file-based micro-batch, etc.) and provides a `scan()` method that returns a `SendableRecordBatchStream` for each micro-batch.

The Flow Event Schema

The fundamental design choice in Sail's streaming is the **flow event schema**. Every record in a streaming plan carries two extra fields prepended to the user's data columns:

```
// crates/sail-common-datafusion/src/streaming/event/schema.rs
pub const MARKER_FIELD_NAME: &str = "_marker";   // Binary, nullable
pub const RETRACTED_FIELD_NAME: &str = "_retracted";  // Boolean, non-nullable

pub fn to_flow_event_schema(schema: &Schema) -> Schema {
    let mut fields = vec![
        Field::new(MARKER_FIELD_NAME, DataType::Binary, true),
```

```
        Field::new(RETRACTED_FIELD_NAME, DataType::Boolean, false),
    ];
    fields.extend(schema.fields().iter().map(|x| x.as_ref().clone()));
    Schema::new(fields)
}
```

- `_marker`: `NULL` for data rows; non-null for control messages (watermarks, checkpoints).
- `_retracted`: `false` for normal INSERT rows; `true` for DELETE/retraction events (used in stateful aggregations with retractions).

The streaming physical plan operates on flow event `RecordBatch`es throughout. Only the final `StreamCollectorExec` strips these fields before writing to the sink. This architecture supports future stateful streaming operations (like retract-mode aggregations) without changing the physical plan execution model.

Streaming Query Lifecycle

`StreamingQuery` in `crates/sail-spark-connect/src/streaming.rs` manages the lifecycle of a running streaming query. It spawns a background tokio task that drives the execution:

```
// crates/sail-spark-connect/src/streaming.rs
pub struct StreamingQuery {
    name: String,
    info: Vec<StringifiedPlan>,       // plan strings for explain()
    error: watch::Receiver<Option<SparkThrowable>>,   // latest error
    stopped: watch::Receiver<bool>,   // has the query stopped?
    signal: Option<oneshot::Sender<()>>,  // stop signal
    awaitable: bool,
}
```

A `watch::Sender/Receiver` pair propagates query state (running, stopped, errored) from the background task to `SparkSession.get_streaming_query_status`. When a Python client calls `query.stop()`, it sends a signal to the `oneshot::Sender`, causing the background loop to exit. The `watch::Receiver` is cloneable, so multiple callers can observe the same state channel.

The `StreamingQueryManager` inside `SparkSession` tracks all active streaming queries by `StreamingQueryId` (a `{query_id, run_id}` pair). It supports `list_active_queries()`, `await_queries()` (blocks until all active queries terminate), and `stop_query()`.

Streaming vs Batch: The Shared Path

Batch and streaming execution share the same `JobRunner` and physical plan execution path below the `rewrite_streaming_plan` step. A streaming plan's physical execution looks like batch execution — the stream runs continuously, micro-batch by micro-batch, through `execute_stream`. The difference is entirely in how the logical plan is structured (flow event schema nodes wrapping the user's plan) and how the `StreamSource` generates RecordBatches (by polling the underlying source repeatedly rather than exhausting a bounded dataset).

Summary

Sail's execution layer is designed around four key abstractions:

1. **`JobRunner` trait** — decouples planning from execution; `LocalJobRunner` and `ClusterJobRunner` are drop-in substitutes.
2. **Actor model** — all mutable execution state lives in actors (`DriverActor`, `WorkerActor`) that process messages sequentially, eliminating lock contention.
3. **`JobGraph`** — a stage-based distributed execution plan derived from the physical plan tree, with explicit input modes and output distributions for each stage boundary.

4. **Streaming execution** — the same `JobRunner` handles streaming, after a logical plan rewrite that wraps user nodes in flow-event-schema adapters; `StreamingQuery` manages lifecycle via `watch` channels.

The execution layer has no knowledge of Spark, Spark Connect, or Arrow Flight. It receives an `Arc<dyn ExecutionPlan>` and produces a `SendableRecordBatchStream`. The planning layer's job is to put the right tree in; the execution layer's job is to run it efficiently.

Chapter 7: Catalog Integrations

What Is a Catalog?

In Spark, a catalog is the metadata service that maps table names to table definitions — schemas, storage locations, file formats, partition layouts. PySpark code like `spark.table("db.orders")` resolves `db.orders` through the catalog to find out where the data is and how to read it.

Sail supports multiple catalog backends, pluggable at configuration time:

Crate	Backend
`sail-catalog-memory`	In-process hash map (default; no persistence)
`sail-catalog-glue`	AWS Glue Data Catalog
`sail-catalog-hms`	Apache Hive Metastore (Thrift)
`sail-catalog-iceberg`	Apache Iceberg REST Catalog
`sail-catalog-unity`	Databricks Unity Catalog (REST)
`sail-catalog-onelake`	Microsoft OneLake / Fabric
`sail-catalog-system`	Built-in system catalog (`spark_catalog`)

All of them implement a single Rust trait. Adding a new catalog backend means implementing one trait and registering it in the session configuration.

The `CatalogProvider` Trait

The trait lives in `crates/sail-catalog/src/provider/mod.rs`:

```
/// A trait that defines the interface for a catalog.
/// A catalog contains *databases*, where each database has a multi-level name
/// that represents a *namespace*.
/// A database contains *objects* such as *tables* and *views*.
#[async_trait::async_trait]
pub trait CatalogProvider: Send + Sync {
    fn get_name(&self) -> &str;

    async fn create_database(&self, database: &Namespace, options:
CreateDatabaseOptions)
        -> CatalogResult<DatabaseStatus>;
    async fn get_database(&self, database: &Namespace) ->
CatalogResult<DatabaseStatus>;
    async fn list_databases(&self, prefix: Option<&Namespace>) ->
CatalogResult<Vec<DatabaseStatus>>;
    async fn drop_database(&self, database: &Namespace, options: DropDatabaseOptions)
-> CatalogResult<()>;

    async fn create_table(&self, database: &Namespace, table: &str, options:
CreateTableOptions)
        -> CatalogResult<TableStatus>;
```

```
    async fn get_table(&self, database: &Namespace, table: &str) ->
CatalogResult<TableStatus>;
    async fn list_tables(&self, database: &Namespace) ->
CatalogResult<Vec<TableStatus>>;
    async fn drop_table(&self, database: &Namespace, table: &str, options:
DropTableOptions) -> CatalogResult<()>;
    async fn alter_table(&self, database: &Namespace, table: &str, options:
AlterTableOptions)
        -> CatalogResult<TableStatus>;

    async fn create_view(&self, database: &Namespace, view: &str, options:
CreateViewOptions)
        -> CatalogResult<TableStatus>;
    async fn get_view(&self, database: &Namespace, view: &str) ->
CatalogResult<TableStatus>;
    async fn list_views(&self, database: &Namespace) ->
CatalogResult<Vec<TableStatus>>;
    async fn drop_view(&self, database: &Namespace, view: &str, options:
DropViewOptions) -> CatalogResult<()>;
}
```

The `Namespace` type is a `Vec<String>` — the multi-part database name (e.g. `["default"]` or `["hive", "prod"]`). `TableStatus` and `DatabaseStatus` are structs that carry the full metadata needed to create a DataFusion table provider (schema, location, format, properties).

Note that all methods are `async`. Catalog operations are inherently I/O-bound: they call remote APIs (Glue, HMS, Unity), and Rust's `async_trait` makes this natural. The `#[async_trait::async_trait]` macro is necessary because Rust does not yet support `async fn` in traits natively at the time of writing.

The In-Memory Catalog

`MemoryCatalogProvider` in `crates/sail-catalog-memory/` is the reference implementation and the default when no external catalog is configured:

```
pub struct MemoryCatalogProvider {
    name: String,
    databases: DashMap<Namespace, MemoryDatabase>,
}

struct MemoryDatabase {
    status: DatabaseStatus,
    tables: HashMap<String, TableStatus>,
    views: HashMap<String, TableStatus>,
}
```

`DashMap` is a concurrent hash map (like `RwLock<HashMap>` but with finer-grained sharding). The top-level `databases` map uses `DashMap` because multiple async tasks may access it concurrently; the inner `tables` and `views` maps are protected by the `DashMap` entry lock.

`create_database` illustrates the idempotent creation pattern used throughout:

```
async fn create_database(
    &self,
    database: &Namespace,
    options: CreateDatabaseOptions,
) -> CatalogResult<DatabaseStatus> {
    let CreateDatabaseOptions { if_not_exists, comment, location, properties } =
```

```
options;
    let entry = self.databases.entry(database.clone());
    match entry {
        Entry::Occupied(entry) => {
            if if_not_exists {
                Ok(entry.get().status.clone())
            } else {
                Err(CatalogError::AlreadyExists(
                    CatalogObject::Database,
                    quote_namespace_if_needed(database),
                ))
            }
        }
        Entry::Vacant(entry) => {
            // ... insert new database
        }
    }
}
```

The `if_not_exists` flag maps to Spark's `CREATE DATABASE IF NOT EXISTS`. This pattern is consistent across all catalog implementations.

The AWS Glue Catalog

`GlueCatalogProvider` in `crates/sail-catalog-glue/` uses the `aws-sdk-glue` Rust crate. Client initialization is lazy — the `OnceCell<Client>` is initialized on the first request, allowing the provider to be constructed cheaply:

```
pub struct GlueCatalogProvider {
    name: String,
    config: GlueCatalogConfig,
    client: OnceCell<Client>,
}

pub(super) async fn get_client(&self) -> CatalogResult<&Client> {
    self.client
        .get_or_try_init(|| async {
            let mut config_loader = aws_config::defaults(BehaviorVersion::latest());
            if let Some(region) = &self.config.region {
                config_loader = config_loader.region(Region::new(region.clone()));
            }
            if let Some(endpoint) = &self.config.endpoint_url {
                config_loader = config_loader.endpoint_url(endpoint);
            }
            let sdk_config = config_loader.load().await;
            Ok(Client::new(&sdk_config))
        })
        .await
}
```

`OnceCell::get_or_try_init` is the standard async lazy-initialization pattern in Tokio: the closure runs at most once; concurrent callers wait for it. This means the AWS credentials are loaded and validated at the first catalog operation, not at startup.

The Glue catalog also handles Iceberg tables stored in Glue (detected by inspecting the table properties for Iceberg markers):

```rust
// crates/sail-catalog-glue/src/iceberg.rs
pub fn is_iceberg_table(table: &aws_sdk_glue::types::Table) -> bool {
    table.parameters()
        .and_then(|p| p.get("table_type"))
        .map(|v| v.eq_ignore_ascii_case("ICEBERG"))
        .unwrap_or(false)
}
```

When the provider detects an Iceberg table, it returns a `TableStatus` that routes the table scan to the Iceberg scan implementation rather than the generic Hive/Parquet scan.

The Hive Metastore Catalog: Thrift Code Generation

HMS uses a Thrift-based RPC protocol (not REST, not gRPC). Sail generates the Thrift client from the `.thrift` IDL file at build time using `volo-build`:

`crates/sail-catalog-hms/build.rs`:

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
    println!("cargo:rerun-if-changed=build.rs");
    println!("cargo:rerun-if-changed=thrift/hive_metastore.thrift");

    volo_build::Builder::thrift()
        .add_service("thrift/hive_metastore.thrift")
        .split_generated_files(true)
        .write()?;

    Ok(())
}
```

The generated client is included into the crate at compile time:

```rust
// crates/sail-catalog-hms/src/lib.rs
pub mod hms {
    #[expect(clippy::allow_attributes)]
    mod internal {
        include!(concat!(env!("OUT_DIR"), "/volo_gen.rs"));
    }
    pub use internal::volo_gen::hive_metastore::*;
}
```

`volo` is a Rust RPC framework from ByteDance that supports both Thrift and gRPC. The build script generates type-safe Rust structs and an async client from the `.thrift` IDL. The `#[expect(clippy::allow_attributes)]` suppresses Clippy warnings in generated code, matching the pattern used in `sail-spark-connect` for protobuf.

The HMS provider supports SASL/Kerberos authentication (common in enterprise Hadoop deployments):

```rust
pub struct HmsCatalogConfig {
    pub uris: Vec<String>,
    pub thrift_transport: Option<String>,
    pub auth: Option<String>,
    pub kerberos_service_principal: Option<String>,
    pub min_sasl_qop: Option<String>,
    pub connect_timeout_secs: Option<u64>,
}
```

Multiple URIs provide high availability: if one metastore is unreachable, the provider tries the next. The `active_index: usize` in `HmsClientState` tracks which endpoint is currently active.

Unity Catalog

Databricks Unity Catalog uses a REST API. `UnityCatalogProvider` in `crates/sail-catalog-unity/` uses `reqwest` for HTTP:

```
pub struct UnityCatalogProvider {
    name: String,
    catalog_config: UnityCatalogConfig,
    client: OnceCell<Client>,
}
```

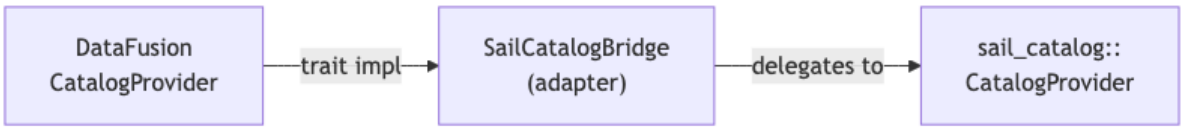
Like the Glue provider, the HTTP client is initialized lazily. Unity Catalog requires an access token; Sail supports configuring it via `UnityCatalogConfig`:

```
impl UnityCatalogProvider {
    const DEFAULT_URI: &'static str = "http://localhost:8080/api/2.1/unity-catalog";
}
```

The Unity Catalog API uses a REST-based namespace hierarchy: catalog → schema → table. Sail maps Spark's two-level namespace (database + table) to Unity's three levels.

The CatalogProvider → DataFusion Bridge

`CatalogProvider` is Sail's trait, not DataFusion's. DataFusion has its own `CatalogProvider` and `SchemaProvider` traits. Sail has an adapter layer in `sail-common-datafusion` that bridges the two:



When DataFusion's query planner needs to resolve `db.orders`, it calls into DataFusion's catalog API. The bridge forwards the call to the Sail `CatalogProvider`, which calls the appropriate backend (Glue, HMS, etc.), gets back a `TableStatus`, and constructs a DataFusion `TableProvider` that knows how to scan the table.

Table providers are constructed per table format: - Parquet, CSV, ORC, Avro, JSON → `sail-data-source` file format providers - Delta Lake → `sail-delta-lake` scan (reads), `DeltaExtensionPlanner` (writes, merge) - Iceberg → `sail-iceberg` scan (read-only; write not yet implemented)

Delta Lake write scope. Only append and overwrite writes are fully operational through the normal write path. The `crates/sail-delta-lake/src/operations/mod.rs` module has `delete`, `update`, `cdc`, `merge`, `optimize` all commented out — they are not yet implemented as standalone Delta operations. DELETE and MERGE work via a separate code path: `ExpandRowLevelOp` rewrites the logical plan to `RowLevelWriteNode`, which routes to `plan_delete`/`plan_merge` in `DeltaPhysicalPlanner`. OPTIMIZE, VACUUM, and Change Data Feed are not supported.

Catalog Error Handling

`CatalogError` is a typed enum:

```
pub enum CatalogError {
    NotFound(CatalogObject, String),      // object type + name
    AlreadyExists(CatalogObject, String),
    InvalidArgument(String),
```

```
    NotSupported(String),
    InternalError(String),
}

pub enum CatalogObject {
    Catalog,
    Database,
    Table,
    View,
    Column,
    Partition,
}
```

`CatalogObject` carries the kind of the missing or conflicting entity, so error messages can say "table 'orders' not found" rather than just "not found". Each implementation converts its own error types (AWS SDK errors, Thrift errors, HTTP errors) into `CatalogError` variants.

Summary

Sail's catalog layer is a thin trait (`CatalogProvider`) with seven implementations. Each implementation handles the idiosyncrasies of its backend: - **Memory**: lock-free concurrent hash maps - **Glue**: AWS SDK, lazy credential initialization, Iceberg table detection - **HMS**: build-time Thrift codegen, HA failover, SASL/Kerberos support - **Unity**: REST API, access token authentication - **Iceberg/OneLake**: format-specific REST APIs

All implementations converge to the same `TableStatus` type, which the DataFusion bridge uses to construct appropriate table providers. A Delta Lake table in Glue, an Iceberg table in HMS, and a Parquet table in the in-memory catalog are all scanned by the same physical plan machinery — only the catalog and table-format layers differ.

Write support summary by format: Parquet/CSV/JSON/ORC/Avro support full read/write. Delta Lake supports read, append/overwrite write, MERGE, and basic DELETE. Iceberg is read-only in the current version. OPTIMIZE, VACUUM, and CDC are not yet available for any format.

Chapter 8: Rust Patterns Throughout Sail

Overview

Sail is a large, production Rust codebase with strict Clippy settings. Reading through it, certain patterns appear repeatedly: a particular approach to error handling, a specific async model, a code generation strategy, and a way of bridging Rust and Python. This chapter collects those patterns in one place.

1. Error Handling with `thiserror`

Sail has one typed error enum per crate. Every crate defines its own `XxxError` and `XxxResult<T>` alias. The workspace-level Clippy configuration bans `unwrap_used` and `expect_used`:

```
# Cargo.toml (workspace)
[workspace.lints.clippy]
unwrap_used = "deny"
expect_used = "deny"
panic = "deny"
```

This forces every error to be handled explicitly. There are no panics in user-triggered paths.

The `thiserror` Pattern

Each crate uses `thiserror` for its error enum:

```rust
// crates/sail-spark-connect/src/error.rs
#[derive(Debug, Error)]
pub enum SparkError {
    #[error("error in DataFusion: {0}")]
    DataFusionError(#[from] DataFusionError),
    #[error("error in Arrow: {0}")]
    ArrowError(#[from] ArrowError),
    #[error("missing argument: {0}")]
    MissingArgument(String),
    #[error("invalid argument: {0}")]
    InvalidArgument(String),
    #[error("not implemented: {0}")]
    NotImplemented(String),
    #[error("internal error: {0}")]
    InternalError(String),
    #[error("analysis error: {0}")]
    AnalysisError(String),
    // ...
}

pub type SparkResult<T> = Result<T, SparkError>;
```

The `#[from]` attribute generates `impl From<DataFusionError> for SparkError` automatically, so `?` works across error type boundaries.

Layered Conversion

When a lower-level crate's error propagates up to a higher-level crate, there is an explicit `From` implementation that maps each variant. This makes the conversion semantics visible and prevents silent information loss:

```rust
impl From<PlanError> for SparkError {
    fn from(error: PlanError) -> Self {
        match error {
            PlanError::DataFusionError(e)    => SparkError::DataFusionError(e),
            PlanError::MissingArgument(msg)  => SparkError::MissingArgument(msg),
            PlanError::InvalidArgument(msg)  => SparkError::InvalidArgument(msg),
            PlanError::NotImplemented(msg)   => SparkError::NotImplemented(msg),
            PlanError::AnalysisError(msg)    => SparkError::AnalysisError(msg),
            PlanError::DeltaTableError(e)    =>
SparkError::InternalError(e.to_string()),
        }
    }
}
```

`DeltaTableError` does not have a corresponding `SparkError` variant, so it is stringified into `InternalError`. This is an explicit choice — it avoids leaking Delta-specific error types into the protocol layer.

Constructor Methods

Each error type provides named constructors for the common variants:

```rust
impl SparkError {
    pub fn todo(message: impl Into<String>) -> Self
{ SparkError::NotImplemented(message.into()) }
    pub fn unsupported(message: impl Into<String>) -> Self
{ SparkError::NotSupported(message.into()) }
    pub fn invalid(message: impl Into<String>) -> Self
```

```
{ SparkError::InvalidArgument(message.into()) }
    pub fn internal(message: impl Into<String>) -> Self
{ SparkError::InternalError(message.into()) }
}
```

`SparkError::todo(...)` is the honest marker for features not yet implemented — a deliberate choice over `todo!()` which would panic. Sail's Clippy configuration bans `todo` macros, so `SparkError::todo(...)` is the escape hatch.

gRPC Error Mapping

At the gRPC boundary, `SparkError` must become `tonic::Status`. The `From<SparkError> for Status` implementation maps error variants to HTTP/gRPC status codes:

```
impl From<SparkError> for Status {
    fn from(error: SparkError) -> Self {
        match error {
            SparkError::MissingArgument(msg)  => Status::invalid_argument(msg),
            SparkError::InvalidArgument(msg)  => Status::invalid_argument(msg),
            SparkError::NotImplemented(msg)   => Status::unimplemented(msg),
            SparkError::NotSupported(msg)     => Status::unimplemented(msg),
            SparkError::AnalysisError(msg)    => {
                // Spark uses a special error detail type for analysis errors
                let mut details = ErrorDetails::new();
                details.set_error_info(/* ... */);
                Status::with_error_details(Code::InvalidArgument, msg, details)
            }
            SparkError::InternalError(msg) => Status::internal(msg),
            // Python errors get special treatment to include traceback
            SparkError::DataFusionError(e) => {
                let python_cause = /* extract Python traceback if available */;
                // ...
            }
            // ...
        }
    }
}
```

Spark clients expect specific error shapes, including `ErrorInfo` gRPC status details for analysis errors. Sail populates these so PySpark's error messages are recognizable.

2. `async`/`await` and Tokio

Sail's entire server is async. The convention is:

- **Protocol layer** (gRPC handlers): `#[tonic::async_trait]` on trait implementations. This is a `proc_macro` that rewrites `async fn` in trait impls into boxed futures, working around Rust's current limitation on `async fn` in traits.
- **Planning**: `async fn` for catalog lookups and UDF resolution (which may require Python calls).
- **Execution**: Tokio tasks spawned via `tokio::spawn`, channels for coordination.

`async_trait` for Catalog

All catalog operations are async because they may call remote APIs:

```
#[async_trait::async_trait]
pub trait CatalogProvider: Send + Sync {
    async fn get_table(&self, database: &Namespace, table: &str) ->
CatalogResult<TableStatus>;
```

```
    // ...
}
```

`#[async_trait]` (from the `async-trait` crate) desugars this to:

```
fn get_table<'life0, 'life1, 'life2, 'async_trait>(
    &'life0 self,
    database: &'life1 Namespace,
    table: &'life2 str,
) -> Pin<Box<dyn Future<Output = CatalogResult<TableStatus>> + Send + 'async_trait>>
```

The `+ Send` bound is critical — it ensures catalog implementations can be used across Tokio threads without data races.

`tokio::select!` for Timeouts and Cancellation

The heartbeat mechanism in the executor uses `select!`:

```
tokio::select! {
    batch = self.stream.next() => Ok(batch.transpose()?),
    _ = tokio::time::sleep(self.heartbeat_interval) => {
        Ok(Some(RecordBatch::new_empty(self.stream.schema())))
    }
}
```

`select!` races two futures and resolves with the first to complete. If the stream produces a batch before the timer fires, the batch wins. If the timer fires first, an empty batch is emitted to keep the connection alive. The timer branch returns `Ok(Some(empty_batch))` — the empty batch is a valid signal that keeps the gRPC response stream from timing out.

Tokio Channels for Actor Communication

All actor communication uses `tokio::sync::mpsc` (bounded, async):

```
const ACTOR_CHANNEL_SIZE: usize = 8;

pub fn spawn<T: Actor>(&mut self, options: T::Options) -> ActorHandle<T> {
    let (tx, rx) = mpsc::channel(ACTOR_CHANNEL_SIZE);
    let handle = ActorHandle { sender: tx };
    // ...
}
```

The small buffer size (8) is intentional: it provides backpressure. If the actor is processing messages slowly, the sender's `await` on `tx.send(...)` will block, propagating backpressure up to the caller. This prevents unbounded memory growth in message queues.

`oneshot` channels are used for request/response patterns:

```
let (result_tx, result_rx) = oneshot::channel();
self.driver.send(DriverEvent::ExecuteJob {
    plan,
    context: ctx.task_ctx(),
    result: result_tx,
}).await?;
let stream = result_rx.await?;
```

This is the async equivalent of a synchronous return value across actor boundaries.

3. The `UserDefinedLogicalNodeCore` Pattern

When adding a new Spark-specific logical plan node to DataFusion, the pattern is:

1. Define a struct with the node's fields.
2. Derive `Clone, Debug, PartialEq, Eq, Hash` — all required by `UserDefinedLogicalNodeCore`.
3. Use `#[derive(Educe)]` from the `educe` crate to suppress derived traits on fields that cannot implement them (e.g. `DFSchemaRef` does not implement `PartialOrd`).
4. Implement `UserDefinedLogicalNodeCore`.

```
#[derive(Clone, Debug, PartialEq, Eq, Hash, Educe)]
#[educe(PartialOrd)]
pub struct RangeNode {
    range: Range,
    num_partitions: usize,
    #[educe(PartialOrd(ignore))]  // DFSchemaRef cannot be ordered
    schema: DFSchemaRef,
}
```

`educe` is a proc-macro crate that allows fine-grained control over derived traits — here, it derives `PartialOrd` for the whole struct while ignoring the `schema` field. Without this, the `#[derive(PartialOrd)]` would fail to compile because `DFSchemaRef` does not implement `PartialOrd`.

4. Code Generation: Protobuf and Thrift

Sail uses two code generation systems:

Tonic/prost for Protobuf (Spark Connect)

`crates/sail-spark-connect/src/lib.rs`:

```
pub mod connect {
    tonic::include_proto!("spark.connect");
    tonic::include_proto!("spark.connect.serde");
    pub const FILE_DESCRIPTOR_SET: &[u8] =
        tonic::include_file_descriptor_set!("spark_connect_descriptor");
}
```

`tonic::include_proto!` expands to `include!(concat!(env!("OUT_DIR"), "/spark_connect.rs"))`. The actual generation is in `build.rs` using `tonic_build`. The build script compiles the `.proto` files during `cargo build`, not at runtime.

volo-build for Thrift (Hive Metastore)

`crates/sail-catalog-hms/build.rs`:

```
volo_build::Builder::thrift()
    .add_service("thrift/hive_metastore.thrift")
    .split_generated_files(true)
    .write()?;
```

`crates/sail-catalog-hms/src/lib.rs`:

```
pub mod hms {
    mod internal {
        include!(concat!(env!("OUT_DIR"), "/volo_gen.rs"));
    }
    pub use internal::volo_gen::hive_metastore::*;
}
```

Both use the `include!(concat!(env!("OUT_DIR"), "..."))` idiom to bring generated code into the module tree. The `cargo:rerun-if-changed=` directives in `build.rs` ensure incremental rebuilds only regenerate the code when the schema files change.

5. PyO3: The Python Bridge

`sail-python` uses PyO3 to expose Rust types to Python. The key types are `SparkConnectServer` (the embedded server) and `SailFlightSqlServer`:

```
// crates/sail-python/src/spark/server.rs
#[pyclass]
pub(super) struct SparkConnectServer {
    #[pyo3(get)]
    ip: String,
    #[pyo3(get)]
    port: u16,
    config: Arc<AppConfig>,
    runtime: RuntimeHandle,
    state: Option<SparkConnectServerState>,
}

#[pymethods]
impl SparkConnectServer {
    #[new]
    #[pyo3(signature = (ip, port, /))]
    fn new(py: Python<'_>, ip: &str, port: u16) -> PyResult<Self> { /* ... */ }

    fn start(&mut self, py: Python<'_>, background: bool) -> PyResult<()> {
        // ...
        if !background {
            let state = self.state()?;
            py.detach(move || state.wait(false))?;
        }
        Ok(())
    }
}
```

The critical line is `py.detach(move || ...)`. When `background=False`, `start` blocks until the server stops. But it must release the Python GIL while blocking, otherwise Python UDFs (which need the GIL to call back into Python) would deadlock. `py.detach` releases the GIL for the duration of the closure, allowing other Python threads to run.

Python UDF Execution

`sail-python-udf` handles Python UDFs. PySpark serializes UDFs using cloudpickle; the serialized bytes are sent over Spark Connect as a `payload` field in the `RegisterFunction` message. Sail deserializes them on the Rust side using PyO3:

```
pub struct PySparkUDF {
    kind: PySparkUdfKind,    // Batch, ArrowBatch, ScalarPandas, ScalarArrow, ...
    payload: Vec<u8>,        // cloudpickle bytes
    input_types: Vec<DataType>,
    output_type: DataType,
    udf: LazyPyObject,       // lazily deserialized Python callable
}
```

`LazyPyObject` holds an `Arc<OnceLock<Py<PyAny>>>`. On the first invocation, it acquires the GIL (`Python::attach`), deserializes the cloudpickle bytes, and stores the resulting `PyAny` in the `OnceLock`. Subsequent invocations reuse the cached object. This amortizes the cloudpickle deserialization cost across batch invocations.

`PySparkUDF` implements DataFusion's `ScalarUDFImpl`:

```
impl ScalarUDFImpl for PySparkUDF {
    fn invoke_with_args(&self, args: ScalarFunctionArgs) -> Result<ColumnarValue> {
        // Convert Arrow arrays to Python objects
        // Call the Python callable
        // Convert results back to Arrow arrays
    }
}
```

The Arrow-to-Python conversion uses `pyo3::types::PyList` and `pyo3::types::PyDict` for Python UDFs, and `pyarrow.RecordBatch` (via `pyo3-arrow`) for Arrow-native UDFs. The return value is an Arrow array, re-entered into the DataFusion compute pipeline.

The GIL and the Tokio Runtime

Python's GIL is a global lock. DataFusion runs on Tokio, which uses a multi-threaded executor. A Python UDF that holds the GIL blocks all other Tokio tasks on that thread from running. Sail handles this by:

1. Running Python UDF evaluation on a dedicated thread pool separate from the main Tokio executor (via `tokio::task::spawn_blocking`).
2. Releasing the GIL with `py.detach` whenever Rust code blocks on I/O.

This prevents Python UDFs from starving network I/O or other query operations.

6. The `SessionExtension` Pattern

The same problem — attaching typed state to DataFusion's `SessionContext` — appears in three places: `SparkSession`, `PlanService`, and `JobService`. Sail solves this with the `SessionExtension` marker trait and a typed accessor:

```
pub trait SessionExtension: Any + Send + Sync {
    fn name() -> &'static str;
}

pub trait SessionExtensionAccessor {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
}

impl SessionExtensionAccessor for SessionContext {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>> {
        self.state()
            .config()
            .get_extension::<T>()
            .ok_or_else(|| DataFusionError::Internal(
                format!("{} extension not found", T::name())
            ))
    }
}
```

This is a simple type-indexed map. The `T::name()` method provides the human-readable name for error messages. Usages look like:

```
let spark = ctx.extension::<SparkSession>()?;
let job_service = ctx.extension::<JobService>()?;
```

The pattern avoids a singleton or global state; each `SessionContext` carries its own extensions, making sessions truly isolated.

7. The `ScalarFunctionBuilder` DSL

The function registry in `sail-plan/src/function/` takes a distinctive approach: functions are not structs implementing a trait — they are closures. The type alias is:

```
// crates/sail-plan/src/function/common.rs
pub(crate) type ScalarFunction =
    Arc<dyn Fn(ScalarFunctionInput) -> PlanResult<expr::Expr> + Send + Sync>;
```

A `ScalarFunction` takes arguments (a `Vec<expr::Expr>`) plus context, and returns a DataFusion `Expr`. This means most Spark functions are expressed as *logical expression trees*, not as physical UDFs. When Sail resolves `abs(x)`, it does not create a new `ScalarUDF` call node — it emits a DataFusion `abs(x)` expression, which the optimizer can reason about, fold constants in, and push down into scans.

`ScalarFunctionBuilder` provides the ergonomic factory methods:

```
// crates/sail-plan/src/function/common.rs
pub struct ScalarFunctionBuilder;

impl ScalarFunctionBuilder {
    pub fn nullary<F, R>(f: F) -> ScalarFunction      // zero args, e.g. pi()
    pub fn unary<F, R>(f: F) -> ScalarFunction        // one arg
    pub fn binary<F, R>(f: F) -> ScalarFunction       // two args
    pub fn ternary<F, R>(f: F) -> ScalarFunction      // three args
    pub fn quaternary<F, R>(f: F) -> ScalarFunction   // four args
    pub fn var_arg<F, R>(f: F) -> ScalarFunction      // variadic
    pub fn binary_op(op: Operator) -> ScalarFunction  // wraps a BinaryExpr operator
    pub fn cast(data_type: DataType) -> ScalarFunction // wraps a cast expression
    pub fn udf<F: ScalarUDFImpl>(f: F) -> ScalarFunction  // wraps a real
ScalarUDFImpl
    pub fn custom<F>(f: F) -> ScalarFunction          // full control: takes
ScalarFunctionInput
}
```

All argument-counted variants use the `ItemTaker` utility trait (`.zero()`, `.one()`, `.two()`, `.three()`, `.four()`) which returns typed errors if the argument count doesn't match. This produces consistent error messages like "expected 1 argument, got 3" without boilerplate per function.

The registration table is a `lazy_static! HashMap<&'static str, ScalarFunction>`. Here is the math function table, showing the three registration styles:

```
// crates/sail-plan/src/function/scalar/math.rs
pub(super) fn list_built_in_math_functions() -> Vec<(&'static str, ScalarFunction)> {
    use crate::function::common::ScalarFunctionBuilder as F;
    vec![
        // Expression-level (zero allocation at invocation time, optimizer-
transparent):
        ("acos",     F::unary(double(expr_fn::acos))),
        ("atan2",    F::binary(double2(expr_fn::atan2))),
        ("pi",       F::nullary(expr_fn::pi)),
        ("e",        F::nullary(eulers_constant)),
        ("ceil",     F::custom(|arg| ceil_floor(arg, "ceil"))),
        ("floor",    F::custom(|arg| ceil_floor(arg, "floor"))),
        ("negative", F::unary(|x| Expr::Negative(Box::new(x)))),

        // Binary operator shorthands:
```

```
        ("%",  F::custom(spark_modulo)),
        ("+",  F::custom(spark_plus)),
        ("-",  F::custom(spark_minus)),
        ("*",  F::custom(spark_multiply)),
        ("/",  F::custom(spark_divide)),

        // Real ScalarUDF (needed when Spark semantics differ from DataFusion):
        ("bin",      F::udf(SparkBin::new())),
        ("bround",   F::udf(SparkBRound::new())),
        ("try_add",  F::udf(SparkTryAdd::new())),
        ("try_div",  F::udf(SparkTryDiv::new())),
        ("rand",     F::udf(Random::new())),
        // ...
    ]
}
```

The three strategies cover different tradeoffs: - `F::unary/binary/...` with DataFusion built-in functions — zero overhead, optimizer-transparent, no physical UDF - `F::custom(closure)` — handles complex argument mapping or conditional expression construction - `F::udf(impl ScalarUDFImpl)` — when Spark's semantics differ from DataFusion's (null handling, overflow behavior, Spark-specific output format)

The final registry is assembled by collecting all category lists:

```
// crates/sail-plan/src/function/scalar/mod.rs
pub(super) fn list_built_in_scalar_functions() -> Vec<(&'static str, ScalarFunction)>
{
    let mut output = Vec::new();
    output.extend(array::list_built_in_array_functions());
    output.extend(bitwise::list_built_in_bitwise_functions());
    output.extend(datetime::list_built_in_datetime_functions());
    output.extend(geo::list_built_in_geo_functions());
    output.extend(hash::list_built_in_hash_functions());
    output.extend(math::list_built_in_math_functions());
    output.extend(string::list_built_in_string_functions());
    output.extend(variant::list_built_in_variant_functions());
    // ... 22 categories total, ~420 entries
    output
}
```

The map is initialized once at startup via `lazy_static!`:

```
// crates/sail-plan/src/function/mod.rs
lazy_static! {
    pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<&'static str, ScalarFunction> =
        HashMap::from_iter(scalar::list_built_in_scalar_functions());
}
```

The `lazy_static!` means the first query pays the initialization cost; all subsequent queries find an already-populated `HashMap<&'static str, ...>` with O(1) lookup.

HLL and theta sketch aggregates (added in #1971) follow the same `AggFunctionBuilder::custom(...)` pattern. The accumulator state is serialized as raw bytes:

```
// crates/sail-function/src/aggregate/hll_sketch.rs
impl Accumulator for HllSketchAggAccumulator {
    fn update_batch(&mut self, values: &[ArrayRef]) -> Result<()> {
        update_hll_sketch_from_array(&mut self.sketch, values[0].as_ref(), /* ...
```

```
*/)?;
        Ok(())
    }

    fn merge_batch(&mut self, states: &[ArrayRef]) -> Result<()> {
        // Merge partial sketch bytes from other partitions (for distributed
execution)
        union_hll_sketches(states[0].as_ref(), /* ... */)?;
        Ok(())
    }

    fn state(&mut self) -> Result<Vec<ScalarValue>> {
        Ok(vec![ScalarValue::Binary(Some(self.sketch.serialize()))])
    }

    fn evaluate(&mut self) -> Result<ScalarValue> {
        Ok(ScalarValue::Int64(Some(estimate_hll_sketch(&self.sketch.serialize(),
"hll_sketch_agg")?)))
    }
}
```

The sketch is serialized to/from `Binary` `ScalarValue`, which DataFusion uses to shuffle partial aggregation state between partitions in distributed mode.

8. The Gold Test Infrastructure

`sail-gold-test` is a systematic Spark compatibility verifier, not just a test runner. The workflow:

1. **Data generation**: Run the `spark-gold-data` binary against a real Spark cluster. It reads Spark's function documentation (generated from `SparkSQLFunctionDocSuite`) — each function's examples — and saves expected SQL/output pairs as JSON files.
2. **Test replay**: The test suite replays each SQL example against Sail and diffs the output against the saved golden data.

The test infrastructure handles schema matching, result ordering, and type coercion. Functions are organized into groups matching Spark's own documentation structure (`array_funcs`, `string_funcs`, `date_funcs`, etc.).

This is how Sail makes concrete claims about Spark compatibility: not "we think these functions work" but "we have replayed Spark's own documentation examples and the output matches".

9. The spec IR: A Clean Internal Boundary

One non-obvious pattern is the existence of `spec::Plan`, `spec::Relation`, `spec::Expr` etc. in `sail-common`. These are Rust enums that mirror the Spark Connect protobuf but are cleanly typed — no `Option<Box<dyn Any>>`, no `oneof` boilerplate.

The motivation is separation of concerns: `sail-spark-connect` knows about protobuf; `sail-plan` knows about DataFusion. Neither should know about the other. The `spec` IR is the language they both speak. `sail-spark-connect` converts protobuf → spec; `sail-plan` converts spec → `LogicalPlan`. The conversion in each direction is contained.

This also makes `sail-flight` natural: it uses `sail-sql-analyzer` to convert SQL text → spec, then hands the same `spec::Plan` to `sail-plan`. Both entry points converge on the same planning code without any shared knowledge of how the spec was produced.

Summary

Sail's Rust patterns are consistent and intentional:

Pattern	Where	Why
`thiserror` error enums + layered `From`	Every crate	Explicit error propagation, no silent conversions
`SparkError::todo()` instead of `todo!()`	`sail-spark-connect`	Unimplemented paths return gRPC `UNIMPLEMENTED`, not panics
`#[async_trait]`	Catalog, gRPC traits	Async fn in traits until native support stabilizes
`tokio::select!`	Executor heartbeat	Timeout + cancellation without blocking
`OnceCell<Client>`	Glue, HMS, Unity	Lazy initialization of remote clients
`LazyPyObject` / `OnceLock`	Python UDFs	Amortize cloudpickle deserialization
`py.detach()`	Python server + UDFs	Release GIL to avoid deadlocks with async Tokio
`include!(concat!(env!("OUT_DIR"), ...))`	Protobuf, Thrift	Build-time codegen, zero runtime cost
`SessionExtension` type-indexed map	Session state	Typed access to per-session context without globals
`ScalarFunctionBuilder` DSL	`sail-plan` function registry	420+ functions as closures, optimizer-transparent, no per-function struct boilerplate
`lazy_static! HashMap` for functions	`sail-plan`	Amortize function registry init; O(1) lookup per query
`AggregateUDFImpl` + binary state	HLL/theta sketch aggregates	Serialize sketch state as `Binary` for distributed partial aggregation
`TreeParser` / `TreeText` proc-macros	`sail-sql-parser`	Derive parsers and unparsers from annotated AST structs
Perfect-hash keyword map (`phf`)	`sail-sql-parser` lexer	O(1) keyword lookup from `build.rs`-generated table of 368 keywords
`spec::Plan` internal IR	sail-common	Clean boundary between protocol and planning layers

Chapter 9: Where Sail Is Going, and How to Navigate the Codebase

Current State

As of version 0.6.3, Sail covers the Spark 3.5.x and Spark 4.x API surfaces that are most commonly used in production PySpark workloads:

- Spark SQL: most DDL/DML, window functions, lateral joins, subqueries
- DataFrame API: virtually complete for the Spark Connect protocol surface
- Python UDFs and UDAFs: row-by-row, batch (PyArrow), Pandas scalar, iterator, grouped map, co-grouped map
- File formats: Parquet, CSV, JSON, ORC, Avro (read and write)
- Table formats: Delta Lake (Variant Shredding, append/overwrite write, MERGE, basic DELETE — optimize/vacuum/CDC not yet), Apache Iceberg (read-only)
- Catalogs: Memory, AWS Glue, Hive Metastore, Apache Iceberg REST, Unity, OneLake
- Streaming: structured streaming write (append mode), streaming query management
- UDTFs: scalar table functions
- HLL and theta sketch aggregate functions
- Geographic types (Geometry, Geography) with GeoArrow encoding

There are also deliberate gaps — places where `SparkError::todo(...)` or `Status::unimplemented(...)` is returned. These include ML pipelines, resource profile commands, compressed operation plans, and some advanced streaming modes. The `todo` markers are intentional breadcrumbs rather than forgotten code.

Where Sail Is Headed

Several areas are actively evolving:

Distributed execution. The cluster mode driver/worker architecture described in Chapter 6 is functional but evolving. Future work includes Kubernetes-native worker lifecycle management, better fault tolerance with task retry policies, and remote stream storage for shuffle spill.

Delta Lake write support. Variant Shredding is merged. Merge-into (upsert) support via `MergeIntoTableCommand` is implemented. Future work: Z-ordering, optimized compaction, Change Data Feed.

Iceberg write support. Currently read-only. Write support is a high-priority item.

Full streaming coverage. Continuous mode streaming and stateful aggregations are partially supported. Watermarking and event-time triggers are areas of active development.

Spark 4.x compatibility. Sail targets both Spark 3.5.x and 4.x simultaneously. As Spark 4.x features land (new catalog APIs, new type system features, `VARIANT` type), Sail adds them. The `GroupedMap`/`CoGroupedMap` iterator UDFs for Spark 4.1.1 were merged in recent commits.

How to Navigate the Codebase

Entry Points

Start here depending on what you want to understand:

Goal	Start
A PySpark query arrives	`crates/sail-spark-connect/src/server.rs:execute_plan`
How a relation is converted to a logical plan	`crates/sail-plan/src/resolver/query/`
How a specific Spark function is implemented	`crates/sail-function/src/`
How a new catalog is added	`crates/sail-catalog/src/provider/mod.rs` (read the trait), then any `crates/sail-catalog-*/`
How Delta Lake tables are scanned	`crates/sail-delta-lake/src/`
How a Python UDF runs	`crates/sail-python-udf/src/udf/pyspark_udf.rs`
How cluster execution works	`crates/sail-execution/src/driver/`
How the Python package embeds the server	`crates/sail-python/src/spark/server.rs`

The `spec` IR Is the Lingua Franca

If you are confused about where a concept lives, find it in `sail-common/src/spec/`. The spec module defines the canonical Rust representation of Spark's plan and expression types. Every data path through Sail passes through this IR. If a new Spark feature involves a new plan node or expression type, it starts here.

Reading `resolve_query_plan`

The largest single function in the resolver is `resolve_query_plan` (or its close neighbors). It is a large `match` over `spec::Relation` variants. When adding support for a new Spark relation type, this is where you add the arm. The pattern is consistent: destructure the spec struct, recursively resolve child plans, build a DataFusion `LogicalPlan` node or a `UserDefinedLogicalNodeCore` extension.

Tests: Gold Tests and Integration Tests

`sail-gold-test` contains golden-file tests that run SQL queries and compare the output against expected results stored in `.json` files. These are the most valuable tests for Spark compatibility: they capture exact output including column names, types, and values.

Integration tests require a running PySpark client connected to a Sail server. They live in `python/pysail/tests/` and can be run with `pytest` after building and installing the Python package.

Unit tests are embedded in the individual crates using the standard `#[cfg(test)] mod tests` pattern.

How to Contribute

Adding a Spark Function

1. Find the function in `crates/sail-function/src/`. Functions are organized by type (scalar, aggregate, window, table) and by module (math, string, array, map, date, etc.).
2. Implement `ScalarUDFImpl` (or `AggregateUDFImpl`, `WindowUDFImpl`) in Rust.
3. Register the function in the session's function registry.
4. Add a gold test that exercises the function with the same inputs as Spark produces.

The function crate has many examples to follow. The `to_csv` and `timestampdiff` functions were added in recent commits and are good recent examples.

Adding a Catalog Backend

1. Create a new crate `sail-catalog-newbackend`.

2. Implement `CatalogProvider` for your struct.
3. Add a `GlueCatalogConfig`-style configuration struct.
4. Register the provider in the session factory in `sail-spark-connect/src/session_manager.rs` (or wherever catalog selection happens based on config).
5. Add integration tests.

Adding a Logical Plan Node

1. Define the node struct in `sail-logical-plan/src/new_node.rs`.
2. Implement `UserDefinedLogicalNodeCore`.
3. Add the physical counterpart in `sail-physical-plan/src/new_node.rs` implementing `ExecutionPlan`.
4. Add the logical → physical conversion in `sail-plan`'s physical planner extension.
5. Add the spec IR type if needed.
6. Add the resolver arm in `sail-plan/src/resolver/query/`.

Understanding a Spark Compatibility Issue

When PySpark produces different output than expected:

1. Enable debug logging to see the `InitialLogicalPlan`, `FinalLogicalPlan`, and `FinalPhysicalPlan` strings.
2. Check the `spec` IR types — is the spec conversion correct?
3. Check `resolve_data_type` — is the type mapping correct?
4. Check the function implementation — does it handle all edge cases (nulls, empty inputs, extreme values)?
5. Compare with `sail-gold-test` for the specific function or operator.

The Broader Vision

Sail's long-term goal is not just to be a Spark replacement. It is to be a general-purpose compute engine that happens to support Spark as its primary protocol. Arrow Flight SQL is the second front — it makes Sail accessible to the BI and analytics ecosystem without requiring PySpark. Future work includes additional protocols (JDBC via the Arrow JDBC driver, DuckDB's ADBC driver).

The distributed execution engine is designed to be independent of any specific protocol. The `JobRunner` trait means the same execution layer serves Spark Connect, Flight SQL, and potentially future protocols. The catalog layer is similarly protocol-independent.

Sail is Apache-2.0 licensed and lives at `github.com/lakehq/sail`. Contributions are welcome. The GitHub issues and community Slack (linked in the README) are the primary coordination channels.

Closing Thoughts

Sail is a demonstration that Rust's performance and safety characteristics, combined with the Arrow and DataFusion ecosystems, are sufficient to build a production-quality query engine that is genuinely faster and cheaper to run than the JVM-based incumbent. The codebase is not small — ~35 crates, hundreds of files — but it is consistently organized. The patterns described in Chapter 8 appear throughout, making new areas of the codebase recognizable once you have read a few.

The Spark API is, in many ways, a good one. The tabular DataFrame API with SQL support has proven itself across thousands of production pipelines. What Sail shows is that the API and the runtime are separable — and that separating them, building the runtime in a systems language with a modern async ecosystem, yields significant gains. Arrow Flight SQL shows that the same compute engine can serve multiple protocols. The arc of the project bends toward a world where "Spark-compatible" means something richer than "runs on the JVM."